Web and Data Engineering

Kapitel 3: Frontend-Technologien — HTML, JavaScript und CSS

Prof. Dr. Michael Granitzer

Ziel dieser Vorlesung

Jede Web-Anwendung, die ein Nutzer sieht und bedient, besteht aus drei Technologien: HTML für Struktur, CSS für Darstellung und JavaScript für Verhalten. Diese Vorlesung vermittelt die **Grundlagen** aller drei — nicht als Programmierkurs, sondern als **Architekturwissen** für Web Engineers.

Leitfragen

- Warum ist semantisches HTML mehr als Markup?
- Wie macht JavaScript den Browser zur Laufzeitumgebung?
- Wie erzeugt CSS aus derselben Struktur unterschiedliche Layouts?
- Wie arbeiten HTML, CSS und JavaScript im Browser zusammen?

Speaker notes

Die drei Frontend-Technologien haben unterschiedliche Stabilitätshorizonte: HTML-Elemente sind seit HTML5 (2014) weitgehend stabil; CSS erhält laufend neue Features (Container Queries, Cascade Layers, :has()), während die Grundprinzipien — Cascade, Box-Modell, Normal Flow — unverändert bleiben; JavaScript-Frameworks wechseln alle fünf bis acht Jahre den Dominant Player (jQuery → Backbone → Angular → React → Server-Components). Das Architekturwissen — DOM-Modell, Event Loop, Rendering-Pipeline — bleibt davon unberührt und transferiert direkt auf das jeweils aktuelle Werkzeug.

HTML, CSS und JavaScript — Zusammenspiel im Browser

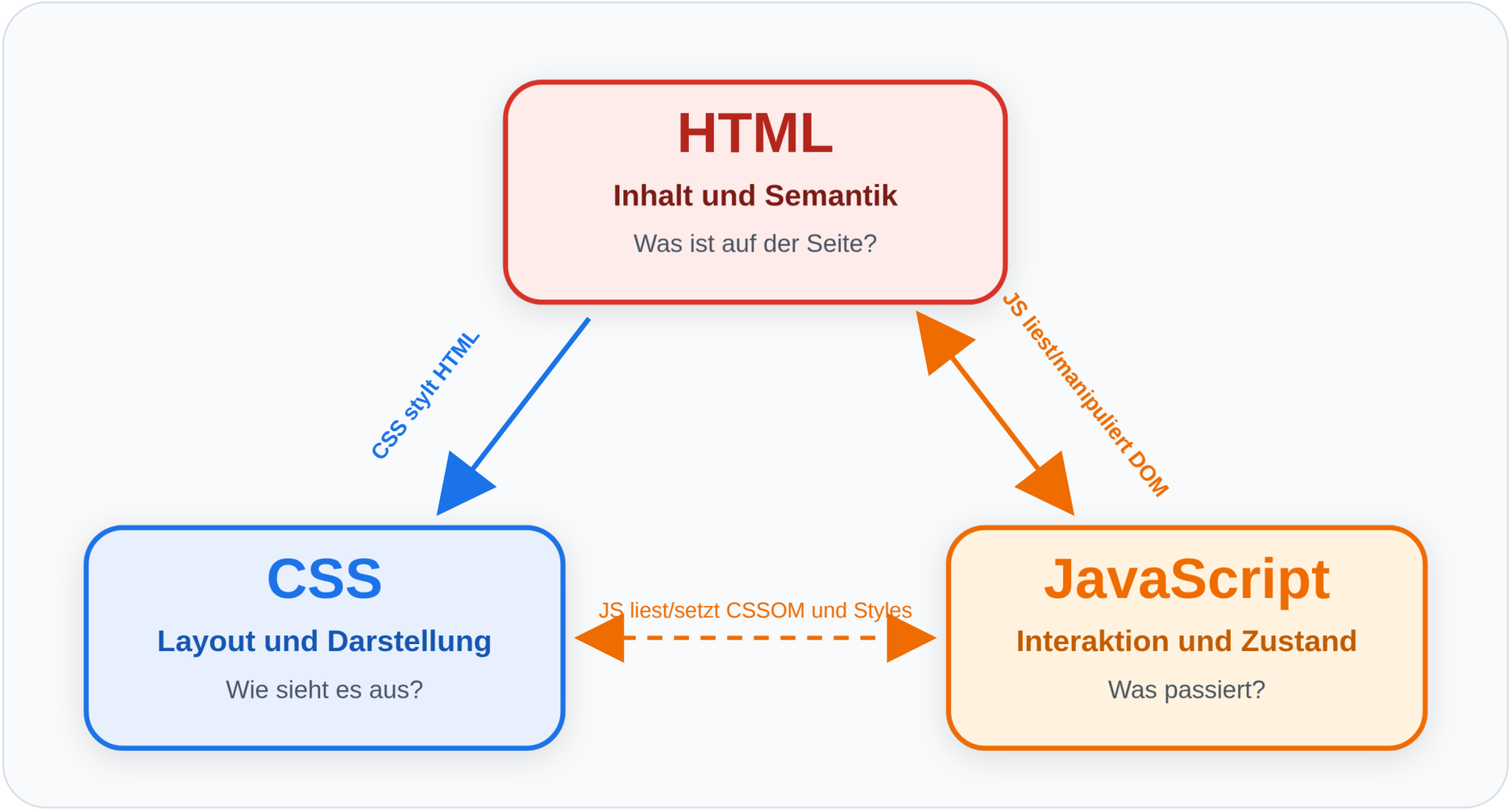
Leitfrage: Welche Aufgabe haben HTML, CSS und JavaScript jeweils im Browser — und wie greifen sie zusammen, bevor wir die drei Technologien einzeln vertiefen?

Speaker notes

Der Browser führt HTML, CSS und JavaScript nicht als drei isolierte Dateien aus, sondern als zusammenhängendes Laufzeitsystem. HTML wird zu einem DOM-Baum, CSS wird zu berechneten Styles, und JavaScript kann diese Strukturen zur Laufzeit lesen und verändern. Diese Kopplung erklärt, warum scheinbar kleine Änderungen an Klassen, Attributen oder DOM-Knoten sichtbare Auswirkungen auf Rendering, Performance und Barrierefreiheit haben.

Das Technologie-Dreieck des Webs

> **Architekturentscheidung:** Inhalt gehört in HTML, Layout in CSS, Interaktion in JavaScript.


HTML
Inhalt und Semantik
Was ist auf der Seite?
CSS stylt HTML
JS liest/manipuliert DOM
CSS
Layout und Darstellung
Wie sieht es aus?
JS liest/setzt CSSOM und Styles
JavaScript
Interaktion und Zustand
Was passiert?

Speaker notes

Diese Zuordnung ist keine Stilfrage, sondern eine Architekturentscheidung. Wenn Inhalt in CSS „versteckt" wird oder Verhalten nur über visuelle Workarounds entsteht, wird das System schwerer verständlich und wartbar. Die Trennung der Schichten unterstützt Barrierefreiheit, progressive Erweiterung und langfristige Pflege.

Hinter den Standards: W3C, WHATWG und die Standardisierung des Webs

Die drei Frontend-Technologien sind keine Produkte einer einzelnen Firma, sondern Ergebnis von Standardisierung.

- **WHATWG** (*Web Hypertext Application Technology Working Group*) ist eine von Browser-Herstellern geprägte Arbeitsgruppe und pflegt heute den **HTML Living Standard** sowie das **DOM**.
- **W3C** (*World Wide Web Consortium*) ist das breitere Standardisierungskonsortium des Webs; dort liegen u. a. **CSS** und **ARIA (Accessible Rich Internet Applications)**. (e.g. Screenreader).
- **ECMA International** standardisiert **JavaScript** unter dem Namen **ECMAScript**.

Wichtig: HTML lag früher auch beim W3C. Heute ist die WHATWG die primäre Instanz für HTML, während das W3C für andere zentrale Webstandards weiter wichtig bleibt. Browser implementieren diese Spezifikationen. Deshalb funktioniert dieselbe Seite grundsätzlich in Chrome, Firefox und Safari.

Speaker notes

Die Standardisierungslandschaft ist seit 2019 neu geordnet: Die WHATWG hat das W3C als primäre HTML-Standardisierungsinstanz abgelöst, nachdem jahrelange Parallelarbeit zu divergierenden Spezifikationen geführt hatte. Die WHATWG ist keine allgemeine Web-Regulierungsbehörde, sondern eine arbeitsnahe Gruppe rund um Browserhersteller und Editorinnen bzw. Editoren, die HTML und DOM als Living Standards kontinuierlich fortschreibt. Das W3C bleibt dennoch zentral, weil dort viele andere Webstandards organisiert werden, insbesondere CSS über die CSS Working Group und ARIA für Accessibility. ECMA International standardisiert ECMAScript (JavaScript) in jährlichen Releases. MDN Web Docs ist die verlässlichste Querschnittsquelle für Browser-Kompatibilität über alle drei Bereiche; `caniuse.com` hilft bei einzelnen Features vor einer Produktionsentscheidung.

Ein Seitenaufruf in drei Schritten

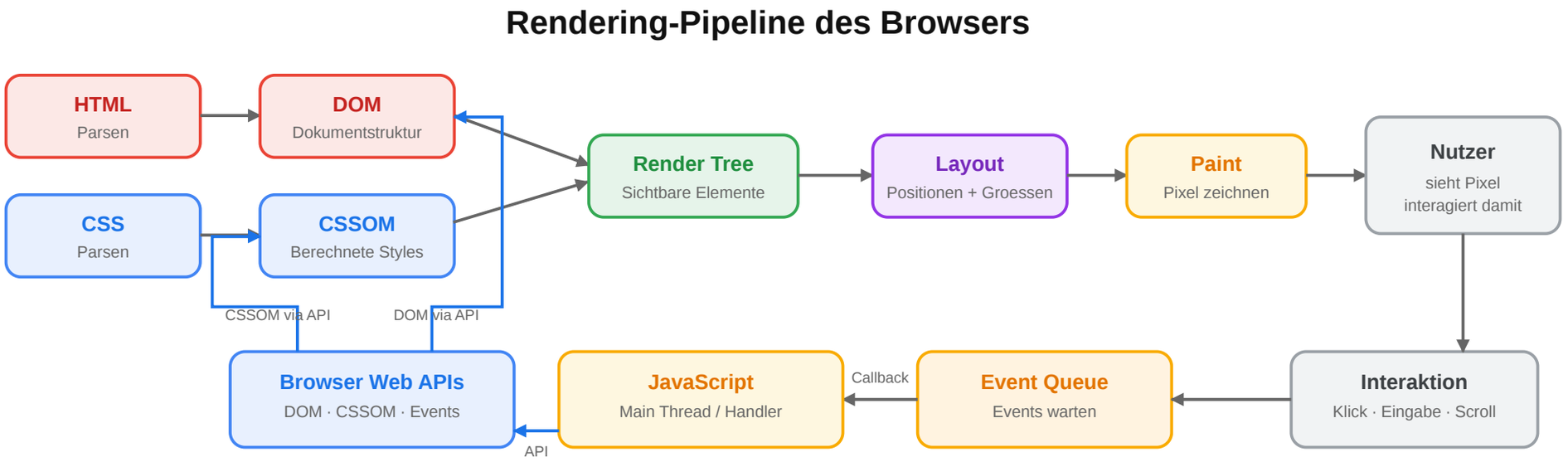
1. **HTML:** Der Browser lädt das Dokument und baut daraus das **Document Object Model (DOM)** auf.
2. **CSS:** Stylesheets (Cascading Style Sheets) werden geladen. Der Browser berechnet daraus, wie der Inhalt aussieht.
3. **JavaScript:** Skripte registrieren Verhalten, reagieren auf Events und können DOM und Styles später verändern.

> **Wichtig:** HTML kommt logisch zuerst. CSS und JavaScript bauen auf dieser Struktur auf.

Speaker notes

Diese Reihenfolge ist auch technisch relevant: CSS-Selektoren benötigen DOM-Knoten als Ziel, und JavaScript arbeitet im Browser meist über Objekte, die aus HTML und CSS abgeleitet wurden. Ein `<script>` im `<head>` kann deshalb auf DOM-Elemente zugreifen, die noch gar nicht geparst wurden, wenn es ohne `defer` zu früh ausgeführt wird. Moderne Seiten laden Skripte daher häufig mit `defer`, damit HTML-Parsing und Script-Ausführung besser koordiniert sind.

Die Rendering-Pipeline des Browsers



Die Pipeline arbeitet auf zwei internen Strukturen: dem **DOM** für HTML und dem **CSSOM** für CSS. Erst daraus erzeugt der Browser den Render Tree und rechnet Layout, Paint und Compositing aus.

Was passiert bei einem Klick?

1. HTML enthält z. B. einen Button und einen Warenkorb-Zähler.
2. CSS definiert, wie Button, Badge und Ladezustand aussehen.
3. JavaScript reagiert auf den Klick.
4. JavaScript ändert DOM oder Klassen.

5. Der Browser rendert die sichtbare Änderung neu.
6. Optional sendet JavaScript asynchron einen Request an den Server.

Speaker notes

Die Rendering-Pipeline ist deterministisch und läuft in sechs Phasen: (1) HTML parsen → DOM, (2) CSS parsen → CSSOM, (3) Render Tree aufbauen (DOM + CSSOM zusammenführen, dabei unsichtbare Elemente ausfiltern), (4) Layout (Berechnung von Position und Größe jedes Elements), (5) Paint (Pixel zeichnen), (6) Composite (Layer zusammenstellen). Bei jeder Änderung am DOM oder CSS fragt der Browser: Wie weit muss ich zurück in die Pipeline? Nur Farbe geändert? → nur Paint + Composite. Position geändert? → Layout + Paint + Composite (Reflow). Nur Transform oder Opacity? → nur Composite (am billigsten). Diese Unterscheidung ist der Schlüssel zu 60-FPS-Animationen. Bei einem Klick entstehen mehrere Zustandswechsel: Der Browser erzeugt ein Event-Objekt, JavaScript verarbeitet dieses Event, der DOM kann verändert werden, und CSS berechnet die Darstellung für den neuen Zustand. Wenn zusätzlich ein Netzwerkrequest ausgelöst wird, laufen UI-Änderung und Serverkommunikation zeitlich auseinander. Dieses Muster ist typisch für moderne Frontends: sofortiges lokales Feedback, danach asynchrone Bestätigung oder Korrektur durch den Server.

Fahrplan: vom Überblick ins Detail

Modul 1: HTML

Semantik, Struktur, Formulare, eingebautes Verhalten

Modul 2: JavaScript

Code im Browser, DOM, Events, Fetch, Alternativen

Modul 3: CSS

Cascade, Layoutmodelle, Grid, Responsive Design

Danach folgt eine **Vertiefung**: Web-APIs und moderne Frontend-Architekturen.

Speaker notes

Die drei Technologien haben unterschiedliche Stabilitätshorizonte. HTML-Semantik ist relativ langlebig, CSS entwickelt sich über Layout- und Selektorfeatures weiter, und JavaScript verändert sich stark über Frameworks und Tooling. Architekturentscheidungen unterscheiden deshalb stabile Browserkonzepte wie DOM, Cascade und Event Loop von kurzlebigeren Frameworkkonventionen.

HTML — Struktur und Semantik

Leitfrage: Warum reicht es nicht, eine Web-Seite einfach mit `<div>`-Elementen zusammenzubauen — und was gewinnt man durch semantisches HTML?

Speaker notes

HTML wurde 1991 von Tim Berners-Lee am CERN für den Austausch wissenschaftlicher Dokumente entworfen und ist bis heute ein Dokumentformat, keine Programmiersprache. Diese Entscheidung ist kein historischer Zufall, sondern der Grund, warum das Web auf jedem Gerät, in jedem Browser und ohne Laufzeitumgebung funktioniert. Die Schichtung Inhalt → Darstellung → Verhalten ist ein Architekturprinzip: HTML-Inhalte sind ohne CSS lesbar, ohne JavaScript navigierbar. Webseiten, die diese Reihenfolge umkehren und JavaScript zur Voraussetzung machen, verlieren Barrierefreiheit, SEO-Sichtbarkeit und Resilience gegenüber Netzwerkproblemen.

Zwei Versionen derselben Seite

Version A

```
<div class="top">
  <div class="logo">Shop</div>
  <div class="links">
    <div><a href="/">Start</a></div>
    <div><a href="/products">Produkte</a></div>
  </div>
</div>
<div class="m">
  <div class="l">
    <div class="t">Funkkopfhörer</div>
    <div>Kabelloser Kopfhörer mit...</div>
    <div class="price">€ 79,99</div>
  </div>
  <div class="r">
    <div>Auch interessant: ...</div>
  </div>
</div>
```

Version B

```
<header>
  <h1>Shop</h1>
  <nav>
    <a href="/">Start</a>
    <a href="/products">Produkte</a>
  </nav>
</header>
<main>
  <article>
    <h2>Funkkopfhörer</h2>
    <p>Kabelloser Kopfhörer mit...</p>
    <p class="price">€ 79,99</p>
  </article>
  <aside>Auch interessant: ...</aside>
</main>
<footer>© 2026 Shop</footer>
```

> **Aufgabe:** Beide Versionen sehen im Browser identisch aus. Was unterscheidet sie — und für wen?

Speaker notes

Für einen sehenden Nutzer mit modernem Browser ist zwischen Version A und B kein visueller Unterschied erkennbar — beide rendern identisch. Der Unterschied wird sichtbar, sobald man die Perspektive wechselt: Ein Screenreader liest `<div>` ohne Kontext, während er `<nav>`, `<article>` und `<footer>` explizit benennt. Eine Suchmaschine kann aus `<article>` + `<h2>` den Hauptinhalt ableiten, aus `<div class="title">` nicht. Ein Browser-Reader-Mode extrahiert `<main>` zuverlässig, `<div class="middle">` nicht. Und ein Entwickler, der nach zwei Jahren zurückkommt, liest semantisches HTML ohne Kommentar — `<div>`-Soup erfordert Reverse Engineering der CSS-Klassen.

Was Version B besser macht

Nutzer	Version A (`<div>`)	Version B (semantisch)
Screenreader	„div, div, div, Link, div, div..." — keine Orientierung	„Navigation mit 2 Links, Hauptinhalt: Artikel ‚Funkkopfhörer', Sidebar"
Suchmaschine	Muss raten, was Hauptinhalt ist	`<article>` + `<h2>` = klares Inhaltsranking
Entwickler	Muss CSS-Klassen lesen, um Struktur zu verstehen	Code ist selbstdokumentierend
Browser	Kein Reader-Mode, keine sinnvolle Druckansicht	Reader-Mode extrahiert `<article>`, Druckansicht setzt `<main>`

Kernprinzip

HTML trennt

- **Struktur** (was ist ein Absatz, eine Überschrift, eine Navigation?)
- von **Darstellung** (wie sieht es aus? → CSS)
- und **Verhalten** (was passiert bei Klick? → JavaScript).

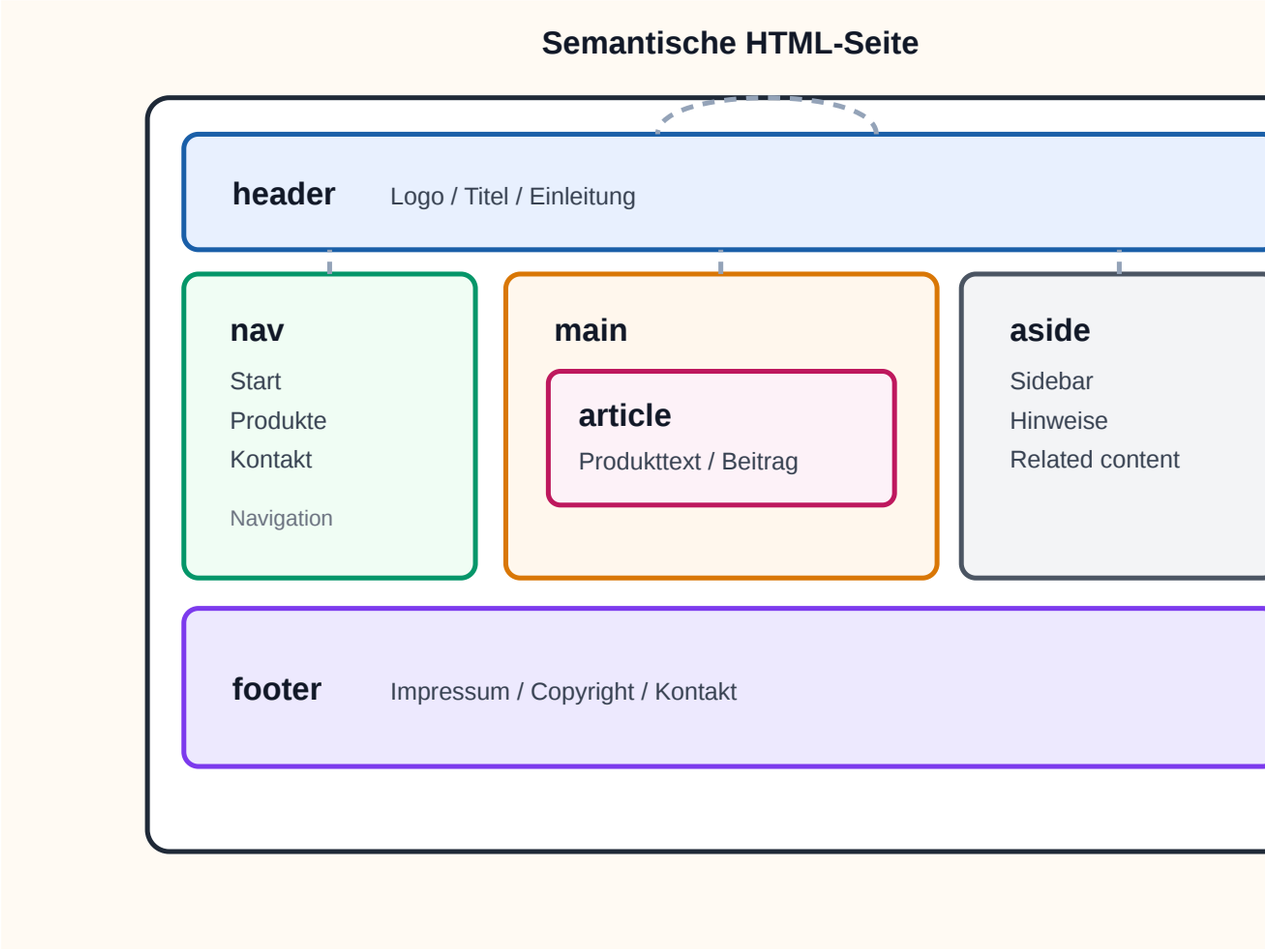
Speaker notes

Zwei der vier Perspektiven haben rechtliche Relevanz: In Deutschland schreibt das Behindertengleichstellungsgesetz (BGG) und die EU-Richtlinie EN 301 549 für öffentliche Einrichtungen barrierefreie Webseiten vor — semantisches HTML ist deren technische Grundlage. Die WCAG-Kriterien 1.3.1 (Info and Relationships) und 2.4.6 (Headings and Labels) sind direkt mit der Wahl zwischen semantischen Elementen und `<div>` verknüpft. Der wirtschaftliche Aspekt: Seiten mit klarer semantischer Struktur erhalten in Googles Qualitätsbewertung höhere Marks für Content-to-Markup-Ratio und erhalten Rich-Snippets häufiger zugewiesen.

Semantische Elemente in HTML5

HTML5 hat generische Container (`<div>`, `<span>`) durch bedeutungstragende Elemente ergänzt:

Element	Bedeutung
`<header>`	Kopfbereich einer Seite oder Sektion
`<nav>`	Navigationsbereich
`<main>`	Hauptinhalt der Seite (einmalig)
`<article>`	Eigenständiger Inhalt (Blogpost, Produkt)
`<section>`	Thematischer Abschnitt
`<aside>`	Ergänzender Inhalt (Sidebar, Hinweis)
`<footer>`	Fußbereich
`<figure>` / `<figcaption>`	Bild mit Beschriftung



Faustregel Wenn ein Element eine **inhaltliche Bedeutung** hat, gibt es dafür ein semantisches HTML-Element. `<div>` ist für Layout-Container ohne eigene Bedeutung.

Speaker notes

HTML5 hat 2014 etwa zwei Dutzend semantische Elemente eingeführt. Die häufig verwechselte Unterscheidung: `<article>` ist eigenständig wiederverwendbar und macht als Fragment außerhalb seiner Seite Sinn (ein Blogpost, ein Produkteintrag, eine Karte in einem Newsfeed); `<section>` ist dagegen nur ein thematischer Abschnitt innerhalb eines größeren Dokuments und ohne diesen Kontext bedeutungslos. `<main>` darf pro Seite nur einmal vorkommen — Browser-Reader-Mode, AMP und mehrere Accessibility-Tools verlassen sich auf diese Einmaligkeit. Screenreader wie NVDA und VoiceOver nutzen Landmark-Elemente (`<nav>`, `<main>`, `<aside>`) für Sprungbefehle, mit denen Keyboard-Nutzer effizient durch die Seitenstruktur navigieren.

Semantik kann mehr: Microformats und Microdata

Semantik in HTML endet nicht bei `<header>` und `<article>`. Für maschinenlesbare Zusatzinformationen gibt es auch Microformats und Microdata.

```html
<a class="h-card" href="/team/max">
  <span class="p-name">Max Mustermann</span>
  <img class="u-photo" src="/img/max.jpg" alt="">
</a>

<article itemscope itemtype="https://schema.org/Person">
  <h1 itemprop="name">Max Mustermann</h1>
  <p itemprop="jobTitle">Data Engineer</p>
</article>
```

- **Microformats** nutzen Klassen wie `h-card`, `p-name`, `u-photo`
- **Microdata** nutzt `itemscope`, `itemtype`, `itemprop`
- Beide ergänzen vorhandenes HTML statt es zu ersetzen

> Details: MDN Microformats und MDN Microdata

Speaker notes

Semantik ist nicht binär: `<header>` und `<article>` sind grobe Struktur-Signale; Microdata mit `schema.org`-Vokabular ist feingranular maschinenlesbare Bedeutung. Google's Rich-Result-Snippets (Sterne-Bewertungen, Preise, Veranstaltungstermine direkt in der Trefferliste) basieren auf `schema.org/Product`, `schema.org/Event` und `schema.org/Review`-Markup. Heute ist JSON-LD in einem `<script type="application/ld+json">`-Tag der von Google empfohlene Mechanismus — es bläht das HTML nicht auf und kann server-seitig einfacher generiert werden. Microformats sind dagegen rückläufig; Microdata bleibt ein valider W3C-Standard, wird aber weniger eingesetzt als JSON-LD.

Wofür braucht man das?

Microformats und Microdata sind nützlich, wenn HTML nicht nur für Menschen lesbar sein soll, sondern auch für Programme, die Inhalte erkennen, zusammenfassen oder weiterverarbeiten.

Microformats

- oft für Personen, Kontakte, Termine, Social Links
- leichtgewichtig und direkt lesbar
- Beispiel: `h-card`, `h-event`, `p-name`

Microdata

- häufig für `Person`, `Product`, `Event`
- enger an Vokabulare wie `schema.org` gebunden
- Beispiel: `itemscope`, `itemtype`, `itemprop`

Typische Nutzung



Was die Nutzung bringt

- bessere maschinelle Erkennbarkeit von Inhalten
- mögliche Rich Results in Suchmaschinen
- weniger Parsing-Hacks in externen Tools
- HTML bleibt die primäre Quelle

- **Suchmaschinen:** Produkte, Personen, Organisationen, Events, Bewertungen
- **Aggregatoren und Crawler:** Inhalte automatisch aus Seiten extrahieren
- **Interne Systeme:** strukturierte Daten ohne separates JSON-Format
- **Semantic Web / schema.org:** zusätzliche Bedeutung direkt im HTML

Kurz gesagt: Microformats/Microdata ergänzen Semantik dort, wo reine Textstruktur nicht reicht.

Speaker notes

Konkretes Beispiel: Ein Online-Shop, der Produkte mit `schema.org/Product` auszeichnet, bekommt in Google-Trefferlisten oft Sterne, Preis und Verfügbarkeit direkt eingeblendet — was die Click-Through-Rate messbar erhöht. Aggregator-Dienste wie Indeed scrapen Stellenanzeigen automatisch von Firmenseiten, sofern diese `schema.org/JobPosting`-Markup enthalten. Wer also strukturierte Inhalte hat (Produkte, Personen, Termine, Rezensionen), gewinnt durch Microdata reale Sichtbarkeit ohne zusätzlichen Code.

schema.org: gemeinsames Vokabular für strukturierte Daten

Wofür?

`schema.org` liefert standardisierte Typen und Eigenschaften, damit Suchmaschinen und andere Systeme wissen, ob ein HTML-Abschnitt eine `Person`, ein `Product`, ein `Event` oder eine `JobPosting` beschreibt.

Wichtige Idee

- **Typ** sagt, was ein Ding ist
- **Properties** beschreiben Details
- dieselben Typen können in `Microdata` oder `JSON-LD` genutzt werden

Typenhierarchie

```
Thing
+-- Person
+-- Organization
|   +-- LocalBusiness
+-- CreativeWork
|   +-- Article
|   +-- Course
+-- Event
+-- Product
```

Beispiele für Properties

- `Person`: `name`, `jobTitle`, `affiliation`
- `Product`: `name`, `offers`, `brand`
- `Event`: `name`, `startDate`, `location`

```
|   +-- Vehicle
+-- Place
    +-- LocalBusiness
```

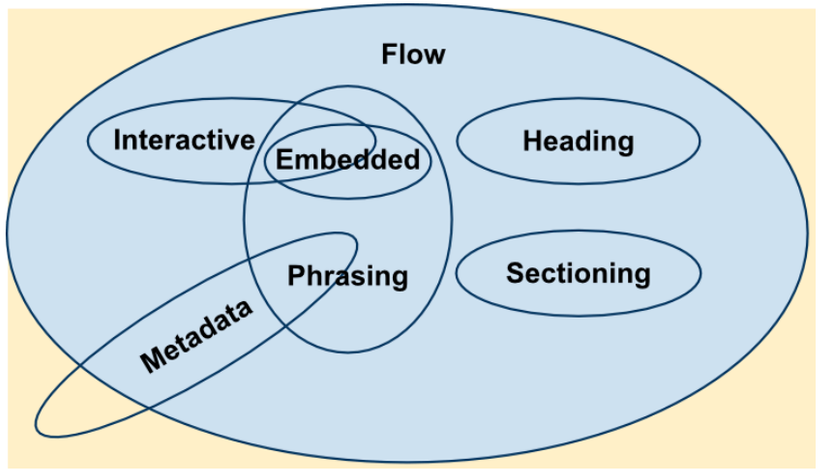
Speaker notes

`schema.org` ist kein neues Datenformat, sondern ein gemeinsames Vokabular. Es beantwortet die Frage: Welche Typen von Dingen gibt es, und welche Eigenschaften gehören typischerweise dazu? `Thing` ist der allgemeine Obertyp; fast alles hängt darunter. Praktisch relevant ist nicht die komplette Ontologie, sondern die Idee der Vererbung: Ein `Product` ist ein `Thing` und kann deshalb allgemeine Eigenschaften wie `name` haben, aber auch produktspezifische wie `offers`. In der Praxis sieht man `schema.org` heute meist als JSON-LD im Head oder Body, nicht mehr als Microdata im sichtbaren HTML. Für das Architekturverständnis ist aber beides dasselbe Prinzip: strukturierte Bedeutung explizit machen.

HTML Content Categories

HTML-Elemente gehören zu formalen Kategorien. Diese Kategorien sagen nicht nur, **was** ein Element bedeutet, sondern auch **wo** es im Dokument eingesetzt werden soll.

Kategorie	Idee	Typische Elemente
Flow Content	allgemeine Bausteine im `<body>`	`<div>`, `<p>`, `<section>`, `<ul>`
Phrasing Content	Inhalt im laufenden Text	`<span>`, `<a>`, `<em>`, `<img>`
Heading Content	Überschriftenstruktur	`<h1>` bis `<h6>`
Sectioning Content	neue inhaltliche Abschnitte	`<article>`, `<section>`, `<nav>`, `<aside>`
Embedded Content	eingebettete externe Inhalte	`<img>`, `<video>`, `<iframe>`
Interactive Content	direkt bedienbare Elemente	`<a>`, `<button>`, `<input>`
Metadata Content	Dokument-Metadaten im `<head>`	`<title>`, `<meta>`, `<link>`



Speaker notes

Content Categories sind die formale Seite der HTML-Semantik. Sie helfen dabei zu verstehen, welche Elemente nur im Textfluss vorkommen, welche ganze Abschnitte bilden, welche interaktiv sind und welche ausschließlich Metadaten beschreiben. Das Venn-Diagramm zeigt, dass viele Elemente mehreren Kategorien gleichzeitig angehören können. Ein `<a>` ist zum Beispiel Phrasing Content und zugleich Interactive Content.

Warum diese Kategorien wichtig sind

Wesentliche Regel

Phrasing Content darf in laufendem Text stehen.
Flow Content ist allgemeiner und darf viele Blockelemente enthalten.
Deshalb ist ein `<span>` in einem `<p>` normal, ein `<div>` in einem `<p>` aber **invalides HTML**.

```
<p>Gültig: <span>Inline-Inhalt</span></p>
<p>Ungültig: <div>Block-Inhalt</div></p>
```

Praktischer Nutzen

- hilft beim **korrekten Verschachteln** von Elementen
- erklärt Validator-Fehler verständlicher
- verhindert stille Browser-Korrekturen
- macht klar, welche Elemente für Text, Abschnitte oder Interaktion gedacht sind

Browser reparieren ungültiges HTML oft stillschweigend. Genau das macht solche Fehler im Layout später schwer nachvollziehbar.

Speaker notes

Phrasing Content umfasst Inhalte, die innerhalb eines laufenden Textes stehen können. Flow Content ist breiter und umfasst auch blockartige Strukturen wie Listen, Abschnitte oder Container. Wenn ein Flow-Element wie `<div>` in einen Paragraphen gesetzt wird, bricht der Browser den Paragraphen implizit ab. Das Ergebnis wirkt oft „fast richtig“, ist aber strukturell falsch und führt zu schwer lesbaren Layout-Bugs.

Formulare: Die Schnittstelle zum Server

Unser Online-Shop braucht ein Registrierungsformular. HTML bietet dafür eingebaute Validierung:

```html
<form action="/api/register" method="POST">
  <label for="email">E-Mail:</label>
  <input type="email" id="email"
         name="email" required>

  <label for="password">Passwort:</label>
  <input type="password" id="password"
         name="password" minlength="8" required>

  <button type="submit">Registrieren</button>
</form>
```

Attribut	Wirkung
`required`	Feld muss ausgefüllt sein
`type="email"`	Browser prüft E-Mail-Format
`minlength`	Mindestlänge des Passworts
`pattern`	Regex-basierte Validierung

> **Diskussionsfrage:** Der Browser validiert clientseitig. Reicht das? Was passiert, wenn jemand das Formular nicht über den Browser, sondern per `curl` oder Fetch API abschickt?

Speaker notes

Clientseitige Validierung ist ein UX-Feature: Sie liefert sofortiges Feedback ohne Server-Roundtrip und vermeidet unnötige HTTP-Requests für offensichtlich ungültige Eingaben. Sie ist aber trivial zu umgehen — über Browser-DevTools, `curl`, Postman oder ein selbstgeschriebenes Skript. Jeder Server muss alle Eingaben unabhängig validieren (Defense in Depth). Das gilt unabhängig davon, ob das Frontend eine HTML-Form, eine React-SPA oder eine native App ist: Der Server-Endpunkt ist die einzige verlässliche Sicherheitsgrenze. SQL-Injection (Vorlesung 6) und XSS/CSRF (Vorlesung 7) sind direkte Konsequenzen fehlender serverseitiger Validierung.

HTML5 als Plattform

HTML5 hat den Browser von einem Dokumentbetrachter zu einer **Laufzeitumgebung** erweitert:

Fähigkeit	API / Element	Typischer Einsatz
Multimedia	`<audio>`, `<video>`	Streaming, Podcasts
Zeichenfläche	`<canvas>`, WebGL	Spiele, Datenvisualisierung
Lokaler Speicher	`localStorage`, `sessionStorage`, IndexedDB	Offline-Daten, Einstellungen
Hintergrundberechnung	Web Workers	CPU-intensive Aufgaben ohne UI-Blockade
Echtzeitkommunikation	WebSockets	Chat, Live-Updates
Geolocation	Geolocation API	Kartenanwendungen
Push-Benachrichtigungen	Notification API + Service Worker	Progressive Web Apps

Architekturentscheidung: Je mehr Fähigkeiten im Browser verfügbar sind, desto mehr Logik **kann** ins Frontend verlagert werden. **Aber sollte sie das?** Ein Online-Shop, der Preise im Client berechnet, riskiert Manipulation. Ein Offline-fähiger Warenkorb dagegen verbessert die User Experience. Die Grenze zwischen Client und Server ist eine bewusste Architekturentscheidung.

Speaker notes

Der moderne Browser ist eine vollständige Anwendungsplattform — vergleichbar mit der Java-Laufzeitumgebung der 2000er oder iOS heute. Progressive Web Apps (PWAs) kombinieren Service Workers (Offline-Cache), Web App Manifest (Installierbarkeit) und Push API (Benachrichtigungen), um native App-ähnliche Erlebnisse ohne App-Store-Distribution zu erreichen. Die strategische Grenzfrage — was gehört in den Browser, was auf den Server, was in eine native App — hängt von Sicherheitsanforderungen (sensible Berechnungen nie im Client), Offline-Anforderungen (Service Worker für Offline-Fähigkeit) und Gerätezugriff (native APIs für Kamera, Bluetooth, NFC nur in nativen Apps vollständig verfügbar) ab.

Responsive Images

Bilder sollten nicht pauschal in der größten Variante ausgeliefert werden. `img` kann dem Browser mehrere passende Ressourcen anbieten:

Beispiel

```
<img
  src="hero-800.jpg"
  srcset="hero-400.jpg 400w, hero-800.jpg 800w, hero-1600.jpg 1600w"
  sizes="(max-width: 768px) 100vw, 50vw"
  alt="Produktfoto eines Kopfhörers">
```

Semantik

- `src` ist der Fallback
- `srcset` beschreibt das Angebot
- `sizes` beschreibt die erwartete Layout-Breite

Entscheidung

Der Browser kombiniert:

- Layout-Breite aus `sizes`
- verfügbare Dateien aus `srcset`
- Pixeldichte des Geräts

Kernidee

Gleiche Bildinformation, andere Auflösung.
Das ist **Resolution Switching**, kein Ersatz für CSS-Layout.

Details: MDN Responsive images

Speaker notes

`src`, `srcset` und `sizes` lösen zusammen das Problem, dass ein Bild im Markup zwar nur einmal vorkommt, aber in verschiedenen Layouts unterschiedlich groß erscheint. `src` ist die robuste Baseline: Wenn der Browser die erweiterten Angaben nicht nutzt, gibt es trotzdem ein Bild. `srcset` mit `w`-Deskriptoren beschreibt reale Quelldateien mit ihren Pixelbreiten. `sizes` beschreibt dagegen nicht die Dateigröße, sondern die erwartete Darstellungsbreite im Layout, zum Beispiel `100vw` auf schmalen Displays und `50vw` auf breiten Displays. Der Browser berechnet daraus die benötigte Ressourcenbreite und berücksichtigt zusätzlich die Device Pixel Ratio, also etwa Retina-Displays mit 2x oder 3x Pixeldichte. Ohne `sizes` nimmt der Browser bei `w`-Deskriptoren häufig `100vw` an, was leicht zu übergroßen Downloads führt. Wichtig ist die Abgrenzung zu `<picture>`: `srcset`/`sizes` wählen meist eine andere Auflösung derselben Bildinformation; `<picture>` wird relevant, wenn Ausschnitt, Komposition oder Format wechseln sollen.

Responsive Images mit `<picture>`

`<picture>` kommt ins Spiel, wenn nicht nur die Auflösung, sondern **Ausschnitt, Komposition oder Format** wechseln sollen:

Beispiel

```
<picture>
  <source
    media="(max-width: 768px)"
    srcset="hero-mobile.webp">
  <source
    type="image/avif"
    srcset="hero-desktop.avif">
  <img
    src="hero-desktop.jpg"
    alt="Produktfoto eines Kopfhörers">
</picture>
```

Details: MDN `<picture>`

Semantik

`<source>` beschreibt mögliche Varianten.
Das innere `<img>` ist das tatsächliche Bild-Element.

Wann?

- anderer Bildausschnitt auf Mobile
- anderes Format wie `AVIF` oder `WebP`

Wie

`img[srcset][sizes]` wählt eine Auflösung.
`<picture>` wählt zuerst die passende Bildvariante.

Fallback

Das innere `<img>` bleibt Pflicht: Es ist Fallback, liefert das `alt`-Attribut und wird verwendet, wenn keine `source`-Regel

greift.

Speaker notes

`<picture>` erweitert `img` um eine Auswahlstufe vor dem eigentlichen Laden des Bildes. Der Browser prüft die `source`-Elemente der Reihe nach und verwendet die erste passende Kombination aus `media`-Bedingung und optionalem `type`. Es gibt zwei Hauptfälle: **Art Direction**, bei der auf kleinen Displays ein anderer Ausschnitt sinnvoll ist, und **Formatverhandlung**, bei der moderne Formate wie `AVIF` oder `WebP` bevorzugt werden können. Das innere `<img>` bleibt semantisch wichtig: Es liefert das eigentliche Bildelement, den Fallback und das `alt`-Attribut. `<picture>` ersetzt `srcset` nicht, sondern ergänzt es. Innerhalb eines `source` kann wiederum `srcset` stehen, um für die gewählte Variante mehrere Auflösungen anzubieten.

Kann HTML das allein? Progressive Enhancement

Bevor man CSS oder JavaScript einsetzt, lohnt die Frage: **Was kann HTML nativ?**

Anforderung	HTML allein?	Erfordert CSS?	Erfordert JS?
Navigation zwischen Seiten	`<a href>` — ja	—	—
Formulardaten an Server senden	`<form action>` — ja	—	—
E-Mail-Format validieren	`<input type="email">` — ja	—	—
Bild mit Beschriftung	`<figure>` + `<figcaption>` — ja	—	—
Dreispaltiges Layout	nein	Grid/Flexbox	—
Hover-Effekt auf Button	nein	`:hover` Pseudo-Klasse	—
Daten vom Server nachladen ohne Seitenreload	nein	nein	`fetch()`
Drag & Drop im Warenkorb	nein	nein	Event Handling

Progressive Enhancement Beginne mit funktionierendem HTML. Füge CSS für Darstellung hinzu. Ergänze JavaScript nur wo nötig. Jede Schicht **erweitert**, statt die vorherige zu ersetzen.

Speaker notes

Progressive Enhancement ist der Gegenentwurf zu „JavaScript-First-Sites": Anstatt JavaScript als Voraussetzung anzunehmen und HTML als Beiwerk zu behandeln, wird die Funktionalität *zuerst* in HTML modelliert. Konsequenz: Die Seite funktioniert auf jeder Plattform, mit jedem Browser, ohne JavaScript — sie ist nur weniger komfortabel. Mit JS-Aktivierung kommen Verbesserungen wie Live-Validierung, Inline-Updates, Animationen. Das Gegenmodell — *Graceful Degradation* — geht den umgekehrten Weg: Es startet mit der vollen JS-Erfahrung und versucht, sie für ältere Browser herunterzubrechen. Beides ist legitim, aber Progressive Enhancement ist robuster, weil HTML der stabilere Baseline ist.

Takeaway

HTML definiert die Struktur und Semantik von Web-Inhalten. Der Unterschied zwischen `<div>`-Suppe und semantischem HTML ist nicht kosmetisch — er bestimmt Barrierefreiheit, Auffindbarkeit und Wartbarkeit. Mit HTML5-APIs wird der Browser zur Plattform, was die Frage aufwirft, wie viel Logik ins Frontend gehört. HTML ist deshalb keine reine Markup-Sprache, sondern die Grundlage der Frontend-Architektur.

Speaker notes

HTML ist das einzige Frontend-Format, das ohne Laufzeitumgebung verarbeitet wird: Suchmaschinen, Screenreader, RSS-Reader, AI-Crawler und Archivierungssysteme (Wayback Machine) lesen HTML direkt. CSS und JavaScript sind für diese Konsumenten optional oder inexistent. Semantisches HTML ist daher kein Komfortmerkmal, sondern die Interoperabilitätsschicht des Webs — der einzige gemeinsame Nenner aller Verarbeitungspipelines, die auf Web-Inhalte zugreifen.

JavaScript — Die Sprache des Webs

Leitfrage: JavaScript ist die einzige Programmiersprache, die nativ in jedem Browser läuft. Was muss man über sie wissen, um Web-Systeme architektonisch zu verstehen?

Speaker notes

JavaScript wurde 1995 von Brendan Eich in zehn Tagen entworfen und hieß ursprünglich Mocha, dann LiveScript, dann JavaScript — ein Marketingentscheid von Netscape, um von Suns Java-Popularität zu profitieren. Technisch sind JavaScript und Java vollständig unverwandt. Das Ausführungsmodell — ein einziger Call Stack, eine Event Queue, ein Event Loop — ist der Grund, warum JavaScript trotz Single-Thread nicht blockiert: I/O-Operationen (fetch, setTimeout, DOM-Events) werden an Browser-APIs delegiert und kehren als Callbacks/Promises in die Queue zurück. Dieses Modell ist identisch in Node.js, weshalb Node denselben Concurrency-Stil auf dem Server ermöglicht.

JavaScript in 60 Sekunden

Was ist JavaScript?

- die **native Programmiersprache des Browsers**
- wird meist über `<script>` oder Bundles ausgeliefert
- arbeitet ereignisgesteuert und kann auf DOM und Web APIs zugreifen

Historischer Kontext

- 1995 bei Netscape entstanden
- entworfen von **Brendan Eich**
- trotz des Namens **nicht** mit Java verwandt

Welche Art Sprache ist das?

- dynamisch typisiert
- multiparadigmatisch
- JIT-kompiliert in modernen Browsern
- für kurze Interaktionen und lange Anwendungen gleichermaßen nutzbar

Rolle im Frontend

JavaScript macht statisches HTML zu einer **interaktiven Anwendung**.

Speaker notes

JavaScript wird im Browser von einer JavaScript-Engine ausgeführt, etwa V8 in Chrome, SpiderMonkey in Firefox oder JavaScriptCore in Safari. Die Sprache selbst wird als ECMAScript standardisiert; DOM, Fetch, Storage oder Canvas gehören dagegen zu den Web APIs der Browserplattform. Diese Trennung erklärt, warum derselbe JavaScript-Sprachkern auch in Node.js läuft, dort aber andere APIs verfügbar sind als im Browser.

Kurzes Beispiel: Verhalten im Browser

```html
<!doctype html>
<html lang="de">
  <head>
    <meta charset="utf-8">
    <title>Warenkorb-Beispiel</title>
  </head>
  <body>
    <button id="buy">In den Warenkorb</button>
    <p id="status">Noch nichts passiert.</p>

    <script>
      const button = document.querySelector('#buy');
      const status = document.querySelector('#status');

      button.addEventListener('click', () => {
        status.textContent = 'Produkt hinzugefügt.';
      });
```

Wann läuft das Skript? Beim Parsen des HTML stoppt der Browser an `<script>`, führt den Inhalt **synchron** aus und parst danach weiter. Weil der Tag *am Ende des `<body>`* steht, existieren `#buy` und `#status` bereits — `querySelector` findet sie. `addEventListener` registriert nur den Callback; der Handler-Code läuft erst, wenn der Browser später ein `click`-Event dispatcht.

Position / Attribut	Wann läuft das Skript?	Risiko
`<script>` am Ende von `<body>` *(hier)*	nach Parsen aller Elemente davor	—
`<script>` im `<head>`, ohne Attribut	sofort, **vor** Body-Aufbau	DOM-Elemente noch nicht da → `querySelector` liefert `null`
`<script defer src=⋯>` im `<head>`	nach Fertigparsen, vor `DOMContentLoaded`	empfohlen für externe Skripte
`<script async src=⋯>` im `<head>`	sobald geladen — Reihenfolge nicht garantiert	nur für unabhängige Skripte (Analytics o. ä.)

> HTML liefert die Elemente. Das `<script>` registriert das Verhalten. Erst der Klick löst die DOM-Änderung aus.

Speaker notes

Das Beispiel ist absichtlich als vollständige HTML-Seite geschrieben, damit Auslieferung und Ausführung sichtbar werden: Der Browser lädt ein HTML-Dokument, baut daraus den DOM auf und führt das eingebettete Script aus. Wichtig zu betonen: Inline-Skripte sind **blockierend** — der Parser hält an, führt das Skript synchron aus und macht erst danach weiter. Externe Skripte (`<script src="…">`) sind ebenfalls blockierend, sofern man nicht `defer` oder `async` setzt; ohne diese Attribute kann ein langsamer Skript-Download die ganze Seite verzögern. `defer` ist die übliche Wahl für Anwendungs-Code: Der Download läuft parallel zum HTML-Parsing, die Ausführung folgt erst nach Fertigparsen — in der Reihenfolge, in der die Tags im Dokument stehen. `async` ist nur für völlig unabhängige Skripte sinnvoll, bei denen die Ausführungsreihenfolge keine Rolle spielt (z. B. Analytics). Eine Alternative zum Skript am Body-Ende ist das `DOMContentLoaded`-Event: `document.addEventListener('DOMContentLoaded', () => { ... })` läuft genau dann, wenn der DOM aufgebaut ist, aber bevor Bilder und Stylesheets vollständig geladen sind. Der Event Listener im Beispiel läuft nicht sofort, sondern erst beim Klick; die Änderung von `textContent` verändert den DOM-Knoten, und der Browser aktualisiert anschließend die sichtbare Darstellung.

Alternative Code-Auslieferung im Browser

JavaScript ist die **native Standardsprache** des Browsers. In der Praxis tauchen daneben oft noch **TypeScript** und **WebAssembly (WASM)** auf.

Modell	Typische Herkunft	Stärken	Grenzen
JavaScript	direkt im Browser ausgeführt	DOM, Events, schnelle UI-Entwicklung, riesiges Ökosystem	dynamische Typisierung kann große Codebasen schwieriger machen
TypeScript	Erweiterung von JavaScript mit Typen	bessere Wartbarkeit, Tooling, Fehler früher sichtbar	läuft nicht direkt im Browser, muss zu JavaScript kompiliert werden
WebAssembly (WASM)	kompiliert aus Rust, C/C++, Go u. a.	hohe Ausführungsgeschwindigkeit, Portierung bestehender Bibliotheken, numerische/mediale Workloads	kein Ersatz für DOM-APIs, meist JavaScript als Host nötig

Einordnung

- **JavaScript bleibt die Orchestrierungsschicht** für UI, Events und Browser-APIs.
- **TypeScript** ist keine Konkurrenz, sondern eine Entwicklungsvariante von JavaScript.
- **WASM ergänzt JavaScript** dort, wo Rechenleistung oder bestehender Native-Code wichtiger sind als DOM-Nähe.
- Typische Beispiele: Bild- und Videobearbeitung, Audio, CAD, Spiele, In-Browser-Datenverarbeitung.

Speaker notes

WASM ist keine „neue Frontend-Sprache", sondern ein binäres Ausführungsformat für eine Sandbox im Browser. Es erweitert das Deployment-Spektrum: Statt Logik nur als JavaScript-Bundle auszuliefern, kann man Teile auch kompiliert bereitstellen. DOM-Manipulation, Event-Handling und die meisten Web APIs bleiben in der Praxis meist bei JavaScript; WASM eignet sich vor allem für rechenintensive Hotspots oder zur Wiederverwendung existierender Bibliotheken aus anderen Sprachen. TypeScript verändert dagegen nicht das Laufzeitmodell, sondern ergänzt den Entwicklungsprozess um statische Typprüfung.

Der DOM: HTML als programmierbare Struktur

Der Browser parst HTML und erzeugt den **Document Object Model (DOM)** — einen Baum, den JavaScript lesen und verändern kann:

DOM nach HTML-Parsing

```
document
└── html
    ├── head
    └── body
        └── article.product
            ├── h2  "Funkkopfhörer"
            ├── p.price  "€ 79,99"
            └── button
                ├── "In den Warenkorb"
                └── span.cart-badge  "0"
```

JavaScript verändert den Baum

```
// Preis aus dem DOM lesen
const price = document
  .querySelector('.price')
  .textContent;

// Badge aktualisieren
const badge = document
  .querySelector('.cart-badge');
badge.textContent = '3';

// Neues Element erzeugen
const msg = document.createElement('p');
msg.textContent = 'Hinzugefügt!';
document.body.appendChild(msg);
```

Kernidee: Der DOM ist die **Brücke zwischen HTML und JavaScript**. Jede Änderung am DOM wird sofort im Browser sichtbar.

Speaker notes

Der DOM ist nicht das HTML-Dokument — er ist der interne Baum, den der Browser aus dem geparsten HTML aufbaut. „View Source" im Browser zeigt das ursprüngliche HTML-Dokument; der DevTools-Inspector zeigt den aktuellen DOM-Zustand, der durch JavaScript beliebig verändert sein kann. Bei einer Single-Page-App können diese beiden vollständig auseinanderlaufen: Der Quelltext enthält nur ein leeres `<div id="root">`, der DOM enthält die vollständige gerenderte Anwendung. Das ist der Kern des SEO-Problems mit clientseitigem Rendering: Crawler, die kein JavaScript ausführen, sehen den leeren Quelltext, nicht den gerenderten DOM. Server-Side Rendering (SSR) löst dieses Problem, indem der initiale DOM serverseitig erzeugt und als vollständiges HTML ausgeliefert wird.

Der DOM: HTML als programmierbare Struktur

Der Browser parst HTML und erzeugt den **Document Object Model (DOM)** — einen Baum, den JavaScript lesen und verändern kann:

DOM nach HTML-Parsing

```
document
└── html
    ├── head
    └── body
        └── article.product
            ├── h2  "Funkkopfhörer"
            ├── p.price  "€ 79,99"
            └── button
                ├── "In den Warenkorb"
                └── span.cart-badge  "0"
```

DOM nach Skript-Ausführung

```
document
└── html
    ├── head
    └── body
        ├── article.product
        │   ├── h2  "Funkkopfhörer"
        │   ├── p.price  "€ 79,99"
        │   └── button
        │       ├── "In den Warenkorb"
        │       └── span.cart-badge  "3"   ← geändert
        └── p  "Hinzugefügt!"              ← neu
```

JavaScript verändert den Baum

```
// Preis aus dem DOM lesen
const price = document
  .querySelector('.price')
  .textContent;

// Badge aktualisieren
const badge = document
  .querySelector('.cart-badge');
badge.textContent = '3';

// Neues Element erzeugen
const msg = document.createElement('p');
msg.textContent = 'Hinzugefügt!';
document.body.appendChild(msg);
```



Kernidee: Der DOM ist die **Brücke zwischen HTML und JavaScript**. Jede Änderung am DOM wird sofort im Browser sichtbar.

JavaScript als Brücke zwischen HTML und CSS

JavaScript orchestriert Verhalten: Es liest HTML-Zustand, verändert den DOM und kann CSS über Klassen oder Custom Properties auslösen.

Drei Strukturen

- **HTML → DOM:** Struktur der Seite
- **CSS → CSSOM:** berechnete Darstellung
- **JavaScript:** liest und ändert beide zur Laufzeit

```
const button = document.querySelector('#buy');
const badge = document.querySelector('.cart-badge');

button.classList.add('loading');  // CSS reagiert
badge.textContent = '3';          // DOM ändert sich

document.documentElement
  .style.setProperty('--cart-count', '3');
```

Abhängigkeit

```
HTML  ← JavaScript → CSS
DOM      Verhalten    CSSOM
```

Konsequenz

CSS übernimmt Darstellung und Animation.
JavaScript setzt nur den Zustand:

HTML und CSS funktionieren in vielen Fällen ohne JavaScript. JavaScript erweitert sie, wenn neuer Zustand oder Serverkommunikation nötig wird.

- Klasse hinzufügen
- Attribut ändern
- DOM-Knoten erzeugen

Speaker notes

CSSOM bedeutet *CSS Object Model*: der interne Baum der CSS-Regeln, den der Browser aus Stylesheets und Inline-Styles aufbaut. Wenn JavaScript Klassen, Inline-Styles oder Custom Properties ändert, beeinflusst es nicht nur HTML, sondern auch die berechnete Darstellung. Diese Aufteilung ist nicht nur Stil, sondern auch Performance: CSS-Transitions und -Animations können vom Browser optimiert und teilweise im Compositor ausgeführt werden, während JavaScript auf dem Hauptthread läuft. Die architektonische Regel lautet daher: JavaScript triggert Zustandswechsel, CSS beschreibt die Darstellung dieses Zustands.

Events: Auf Nutzeraktionen reagieren

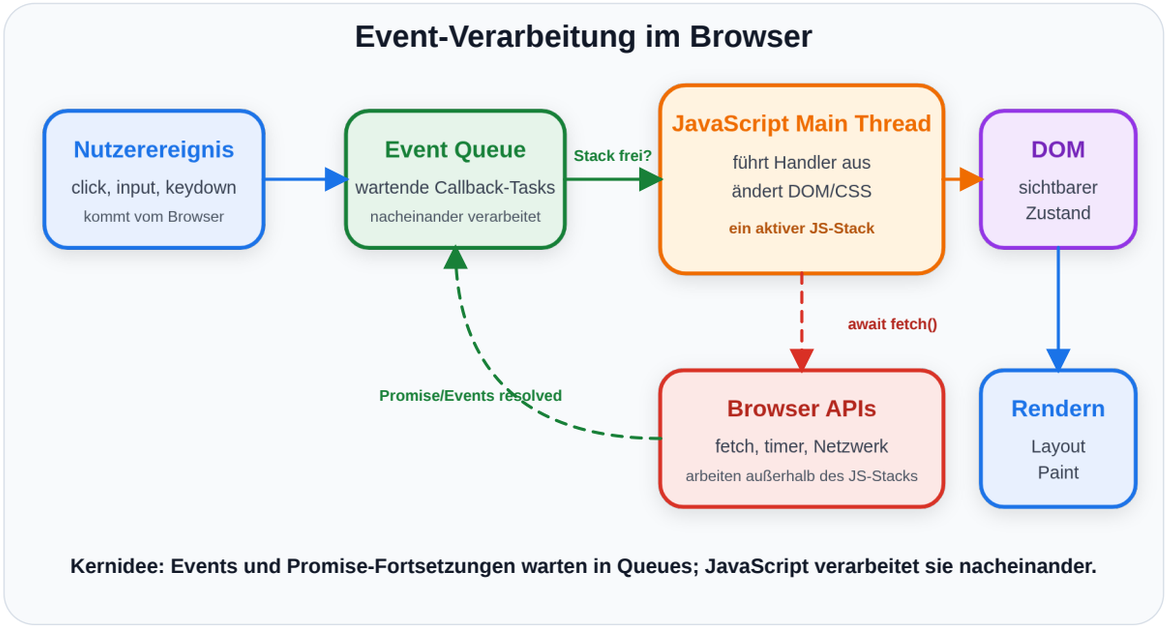
JavaScript arbeitet **ereignisgesteuert**: Code wird ausgeführt, wenn etwas passiert. In unserem Online-Shop:

```
const button = document.querySelector('#add-to-cart');

button.addEventListener('click', async (event) => {
  button.textContent = 'Wird hinzugefügt...';
  button.disabled = true;

  const response = await fetch('/api/cart', {
    method: 'POST',
    body: JSON.stringify({ productId: 42, quantity: 1 })
  });

  if (response.status === 201) {
    button.textContent = '✓ Im Warenkorb';
    updateCartBadge();
  }
});
```



Diskussionsfrage: Der Button wird sofort deaktiviert und der Text geändert, **bevor** die Serverantwort eintrifft. Warum macht man das? Was passiert, wenn die Netzwerkverbindung abbricht?

Speaker notes

JavaScript im Browser läuft für UI-Code typischerweise auf dem Main Thread. Das bedeutet nicht, dass der Browser nur einen Thread hat: Netzwerk, Timer, Rendering-Teile und andere Browser-APIs können parallel oder außerhalb des JavaScript-Call-Stacks arbeiten. Der Event Loop sorgt dafür, dass fertige Ereignisse und Promise-Fortsetzungen nacheinander wieder auf den Main Thread gelangen. Genau deshalb blockiert `await fetch(...)` nicht die gesamte Seite: Die Netzwerkarbeit läuft außerhalb des JavaScript-Handlers, und der Handler wird später fortgesetzt. Das **optimistische UI-Update** ist ein etabliertes Muster für wahrgenommene Performance: Statt auf die Serverantwort zu warten, aktualisiert der Client sofort seinen lokalen Zustand und zeigt Feedback. Wenn der Server fehlschlägt, muss dieser Zustand zurückgerollt werden.

XMLHttpRequest: Der Vorgänger

Vor der Fetch API war `XMLHttpRequest` (XHR) die einzige Möglichkeit, Daten asynchron vom Server nachzuladen — die technische Grundlage für **AJAX** (Asynchronous JavaScript and XML) ab ca. 2005.

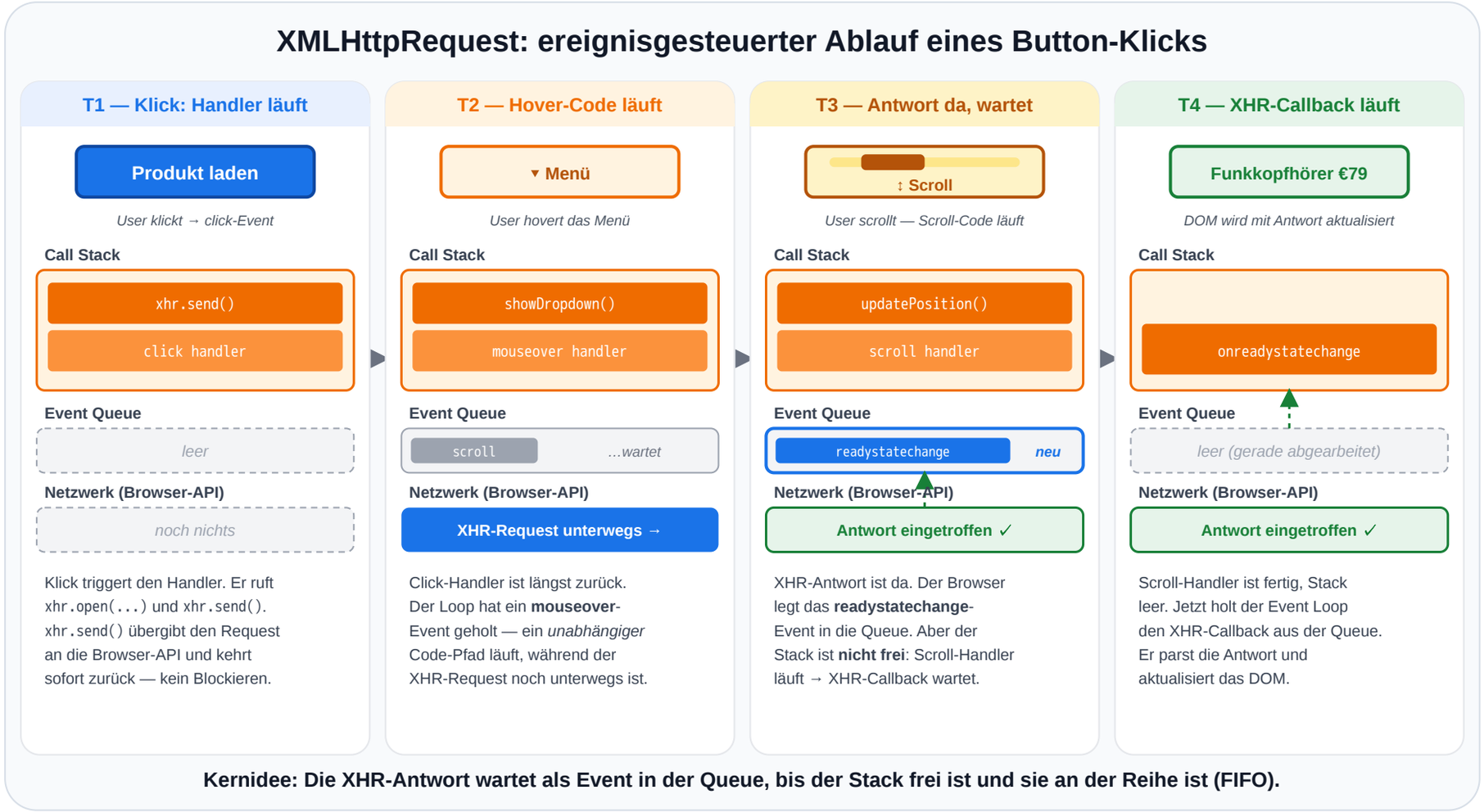
```
// GET-Request mit XMLHttpRequest
const xhr = new XMLHttpRequest();
xhr.open('GET', '/api/products/42');
xhr.onreadystatechange = function () {
  if (xhr.readyState === 4) {              // 4 = DONE
    if (xhr.status === 200) {
      const product = JSON.parse(xhr.responseText);
      renderProduct(product);
    } else {
      showError('Fehler: ' + xhr.status);
    }
  }
};
xhr.onerror = function () { showError('Netzwerkfehler'); };
xhr.send();
```

Schwäche von XHR	Konsequenz
Callback-basiert, kein Promise	Verschachtelung bei mehreren Requests („Callback Hell")
Komplexe State-Maschine (`readyState` 0–4)	Boilerplate, fehleranfällig
Konfiguration verteilt auf Properties + Methoden	Kein flüssiger, deklarativer Aufruf
JSON manuell mit `JSON.parse` parsen	Mehr Schritte, mehr Fehlerquellen

Speaker notes

`XMLHttpRequest` wurde ursprünglich von Microsoft für Outlook Web Access (1999) als ActiveX-Komponente entwickelt und später von allen Browsern übernommen. Trotz des Namens hat das XML-Format nie eine zentrale Rolle gespielt — JSON setzte sich schnell als Datenformat durch. AJAX als Konzept (Jesse James Garrett, 2005) markierte den Übergang von klassischen Multi-Page-Anwendungen zu interaktiven Single-Page-Anwendungen wie Google Maps oder Gmail. Die API ist immer noch in jedem Browser verfügbar und wird von älteren Bibliotheken (jQuery `$.ajax`) intern verwendet, gilt aber für neuen Code als überholt. Konzeptionell wichtig bleibt: `readyState 4` plus `status 200` bedeutet erfolgreich — diese Trennung zwischen *Übertragung abgeschlossen* und *Server hat OK geliefert* findet sich auch in der Fetch API wieder (`response.ok` vs. Netzwerk-Exception).

XHR im Ablauf: Antwort kommt erst beim Event — dazwischen läuft anderer Code



Speaker notes

Die Skizze macht drei Punkte am XHR-Beispiel sichtbar. **Erstens:** Das Ergebnis der HTTP-Anfrage wird *erst beim Event* (`readystatechange` mit `readyState === 4`) verarbeitet — nicht synchron in der Zeile mit `xhr.send()`. Letzteres kehrt sofort zurück und tut nichts mehr; der Browser merkt sich nur, dass `xhr.onreadystatechange` registriert ist, und ruft die Funktion später auf. **Zweitens — das ist der Kern des ereignisgesteuerten Modells:** *Zwischen* dem Absenden und dem Eintreffen der Antwort läuft der Main Thread normal weiter. T2 zeigt das mit einem Hover-Event über das Menü: Während der XHR-Request unterwegs ist, dispatcht der Browser das mouseover-Event, der Event Loop holt den Hover-Handler auf den Stack, eine Dropdown-Funktion läuft. **Drittens — das ist neu in T3:** Selbst *wenn* die XHR-Antwort eintrifft, läuft sie nicht sofort. Der Browser legt das `readystatechange`-Event in die Event Queue. Wenn dort schon ein anderes Event auf seine Verarbeitung wartet (im Beispiel: ein `scroll`-Event, das während der Wartezeit aufgelaufen ist und gerade vom Loop aktiv abgearbeitet wird), muss der XHR-Callback hinten anstehen — die Queue ist FIFO. Erst in T4, nachdem der Scroll-Handler vom Stack zurückgekehrt ist und der Stack frei ist, wird der XHR-Callback auf den Stack gelegt und ausgeführt. Genau dieses Muster — kurzer JS-Block kickt etwas an → Browser arbeitet → das Ergebnis wird als Event in die Queue gestellt → der Loop arbeitet es ab, *wenn* es an der Reihe ist — ist die Grundlage *jeder* asynchronen API im Browser, von `setTimeout` über `addEventListener` bis hin zu `fetch` (nur dass dort statt eines Callbacks ein Promise verwendet wird).

JavaScript-Ökosystem: Überblick

Bereich	Technologie	Rolle
Browser-Frameworks	React, Vue.js, Angular, Svelte	Deklarative, komponentenbasierte UI
Server-Runtime	Node.js, Deno, Bun	Dasselbe Event-Loop-Modell serverseitig
Full-Stack Frameworks	Next.js, Nuxt.js, SvelteKit	Server-Rendering + Client-Interaktion
TypeScript	Typisierte Obermenge von JS	Statische Typprüfung für große Codebases
Build-Tools	Vite, webpack, esbuild	Bündelung, Transpilation, Optimierung

Die Wahl des Frameworks ist eine **Architekturentscheidung** — sie bestimmt, wie Zustand verwaltet, Code organisiert und die Anwendung deployt wird.

Speaker notes

Das JavaScript-Ökosystem ist riesig und ändert sich schnell, aber die Hauptkategorien bleiben relativ stabil: Browser-Frameworks für UI, Server-Runtimes wie Node.js, Full-Stack-Frameworks für Server und Client, TypeScript als Typsystem und Build-Tools für die Auslieferung. Die Wahl ist nie technisch neutral: Sie bestimmt, wie Teams arbeiten, welche Bibliotheken verfügbar sind, wie Deployment funktioniert und wie hoch die Einstiegskosten für neue Entwicklerinnen und Entwickler sind.

Takeaway

JavaScript ist die **Verhaltensschicht** des Webs und zugleich der Orchestrator: Es ist die einzige Technologie, die sowohl DOM (HTML) als auch CSSOM (CSS) verändern kann. Das Ausführungsmodell — single-threaded mit Event Loop — ermöglicht responsive UIs trotz Netzwerk-Latenz. Die Fetch API verbindet Frontend mit Backend. Für Web Engineering sind drei Konzepte entscheidend: das **Event-Loop-Modell**, die **Abhängigkeitsrichtung** (JS erweitert HTML/CSS, nicht umgekehrt) und die Trennung zwischen **Sprachkern** und **Browser-APIs**.

Speaker notes

Das Event-Loop-Modell ist das einheitliche Erklärungsmodell für scheinbar unzusammenhängende JavaScript-Verhaltensweisen: warum der Browser trotz single-threaded Ausführung nicht einfriert (asynchrone Callbacks warten in der Queue, nicht im Stack), warum `async/await` syntaktischer Zucker über Promises ist und keine echte Parallelität erzeugt, warum eine synchrone Schleife die UI blockiert (der Event Loop kann keine anderen Callbacks verarbeiten, solange der Stack besetzt ist), und warum Node.js tausende gleichzeitige Netzwerkverbindungen mit einem einzigen Thread bedienen kann (I/O ist nie blocking — die Callbacks warten in der Queue). DOM, Events, Fetch und Frameworks bauen alle auf diesem Modell auf.

CSS — Layout und Responsive Design

Leitfrage: Warum braucht das Web eine eigene Sprache für Darstellung — und wo endet die Verantwortung von CSS, wo beginnt die von JavaScript?

Speaker notes

CSS ist die anspruchsvollste der drei Frontend-Sprachen, weil sie vollständig deklarativ ist: Die Entwicklerin beschreibt Constraints, der Browser löst sie auf — der konkrete Pixel-Wert ist nie direkt spezifizierbar. Die meisten schwer debugbaren Bugs in Frontend-Code sind keine JavaScript-Fehler, sondern unverstandene CSS-Spezifität, Cascade oder Layout-Wechselwirkungen. Die Kaskade, das Spezifitätssystem und das Box-Modell sind keine Details, sondern die Kernmechanik — wer sie versteht, kann jedes CSS-Framework schnell lesen und anpassen, weil alle Frameworks auf denselben Browser-Grundlagen aufbauen.

CSS in 60 Sekunden

CSS steht für Cascading Style Sheets und beschreibt, **wie HTML dargestellt wird**: Farben, Abstände, Schrift, Layout, Zustände und Responsivität.

Regel

```css
p.highlight {
  color: orange;
  font-weight: 600;
}
```

Semantik

- **Selektor**: `p.highlight` wählt Elemente aus
- **Deklaration**: `color: orange` setzt Stil
- **Regelblock**: alles zwischen `{ ... }`

Browser entscheidet

- Kaskade: Welche Regel gilt?
- Spezifität: Welche Regel ist stärker?
- Vererbung: Was wandert zu Kindern?

Kernidee

CSS ist **deklarativ**: Man beschreibt Regeln und Constraints. Der Browser berechnet daraus konkrete Pixel.

Speaker notes

CSS ist keine Programmiersprache im imperativen Sinn. Es gibt keine Anweisung „zeichne bei x=200, y=300", sondern Regeln, die auf Elemente matchen. Der Browser löst anschließend Konflikte über Kaskade, Spezifität, Ursprung und Reihenfolge auf. Vererbung ist davon getrennt: Eigenschaften wie `color` oder `font-family` wandern von Eltern zu Kindern, Eigenschaften wie `margin`, `padding` oder `border` nicht. Genau dieses Regelmodell macht CSS leistungsfähig, aber auch fehleranfällig, wenn man nicht versteht, warum eine Regel gewinnt.

Warum CSS als eigene Technologie?

Dasselbe HTML soll in verschiedenen Kontexten unterschiedlich aussehen: Desktop, Mobile, Print, Dark Mode, reduzierte Bewegung.

Ohne CSS

- Darstellung steckt im HTML
- Redesign betrifft viele Dateien
- Layout wird mit Struktur vermischt
- Gerätevarianten werden dupliziert

Mit CSS

- HTML bleibt Struktur und Semantik
- Stylesheet kann viele Seiten steuern
- Medien und Zustände sind abbildbar
- Screenreader lesen die Struktur, nicht das Layout

CSS trennt **Bedeutung** von **Darstellung**. Das ist keine Kosmetik, sondern eine Architekturentscheidung.

Speaker notes

Vor CSS wurden Webseiten mit `<font>`, `<center>`, Layout-Tabellen und Inline-Attributen wie `bgcolor` gestaltet. Das führte zu schlecht wartbaren Seiten: Jede Darstellungsänderung musste im HTML gesucht und angepasst werden. CSS wurde 1996 spezifiziert, um Darstellung von Struktur zu trennen. Dadurch kann dasselbe HTML auf dem Bildschirm anders aussehen als im Druck, mobil anders als auf Desktop und im Dark Mode anders als im Light Mode. Barrierefreiheit profitiert ebenfalls, weil semantische HTML-Struktur nicht durch rein visuelle Layouttricks ersetzt werden muss.

Was CSS ohne JavaScript kann

CSS reagiert auf viele Zustände selbst. JavaScript ist erst nötig, wenn neuer Zustand erzeugt oder externe Daten verarbeitet werden.

CSS reicht

- Hover: `button:hover`
- Fokus: `input:focus`
- Formularstatus: `:valid`, `:invalid`
- Geräte-/Theme-Kontext: `@media`
- Übergänge: `transition`, `animation`

JavaScript nötig

- Daten vom Server laden
- DOM-Elemente erzeugen
- Drag & Drop sortieren
- Zustand berechnen oder speichern
- Klassen als Auslöser setzen

Faustregel: CSS beschreibt Darstellung und Übergänge. JavaScript erzeugt oder verändert Zustand.

Speaker notes

CSS kann mehr Interaktion abdecken, als oft angenommen wird: Hover- und Fokuszustände, Formularvalidierung, Media Queries, Container Queries, Dark Mode und einfache Übergänge benötigen kein JavaScript. Auch `<details>`/`<summary>` kann mit CSS gestaltet werden und liefert ein zugängliches Akkordeon ohne eigenen JS-State. JavaScript wird dann nötig, wenn Zustand aus externen Quellen entsteht oder aktiv verändert werden muss: Netzwerkdaten, Warenkorbstatus, dynamisch erzeugte DOM-Knoten, Drag & Drop oder persistente Speicherung.

Welche Farbe hat der Text?

```html
<p id="info" class="highlight">Willkommen im Shop</p>
```

```css
p            { color: blue; }
.highlight   { color: green; }
#info        { color: red; }
p.highlight  { color: orange; }
```

Aufgabe: Der Paragraph matcht alle vier Selektoren. Welche Farbe wird angezeigt — und warum?

Speaker notes

Bei CSS entscheidet nicht nur die Reihenfolge im Stylesheet. Spezifität hat Vorrang; die Reihenfolge ist erst der Tiebreaker, wenn zwei Regeln identische Spezifität haben. Das Spezifitätssystem ist deterministisch: `(1,0,0)` (ID) schlägt immer `(0,99,99)` (Klassen + Elemente). Ein typisches Bug-Muster entsteht, wenn eine neue Regel korrekt formuliert ist, aber nicht greift, weil eine existierende Regel mit höherer Spezifität gewinnt. Der DevTools-Inspector zeigt solche überschriebenen Eigenschaften durchgestrichen an.

Auflösung: Spezifität entscheidet

Selektor	Spezifität (ID, Klasse, Element)	Farbe
`p`	(0, 0, 1)	blue
`.highlight`	(0, 1, 0)	green
`p.highlight`	(0, 1, 1)	orange
`#info`	(1, 0, 0)	**red**

Was bedeutet `(0, 1, 1)`?

Die Werte zählen Selektor-Bestandteile:

- erste Zahl: **IDs**
- zweite Zahl: **Klassen/Pseudo-Klassen**
- dritte Zahl: **Elemente**

Beispiel: `p.highlight` = 0 IDs, 1 Klasse, 1 Element → `(0, 1, 1)`.
Verglichen wird von links nach rechts.

ID schlägt Klasse schlägt Element. In großen Projekten führen Spezifitätskonflikte zu unerwartetem Verhalten — deshalb setzen moderne Projekte auf Konventionen (BEM) oder komponentenbezogenes CSS (Scoped Styles).

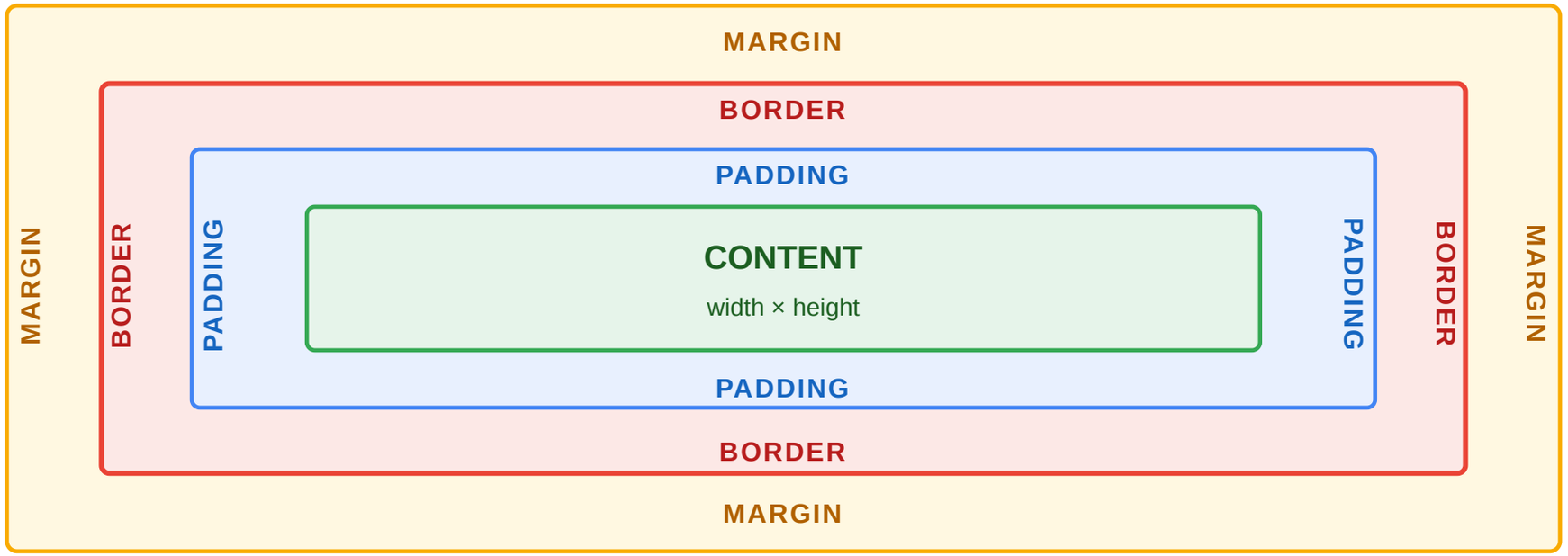
Speaker notes

Die Spezifität wird als Tripel (ID, Klasse, Element) berechnet und lexikografisch verglichen: `(1, 0, 0)` schlägt `(0, 99, 99)`. Inline-Styles (`style="..."`) haben die noch höhere Spezifität `(1, 0, 0, 0)`. `!important` überschreibt alles — und ist deshalb verpönt, weil es die Kaskade außer Kraft setzt und spätere Überschreibungen unmöglich macht. BEM (Block-Element-Modifier) ist eine Namenskonvention, die Spezifitätskonflikte vermeidet, indem jede Komponente mit nur einer Klasse gestylt wird. Scoped Styles in Vue oder CSS Modules in React lösen dasselbe Problem automatisch, indem sie Klassennamen pro Komponente eindeutig machen.

CSS-Layouts: Das Box-Modell

Wie werden HTML-Elemente überhaupt als Layout dargestellt?
CSS betrachtet jedes Element als Box: **Content** → **Padding** → **Border** → **Margin**.

Die häufigste Fehlerquelle: `width: 200px` bedeutet standardmäßig nur den Content — Padding und Border kommen **obendrauf**. Erst `box-sizing: border-box` macht `width` zum Gesamtmaß.



Speaker notes

Das Box-Modell ist das erste wirklich nicht-intuitive Detail von CSS: `width: 200px` + `padding: 20px` + `border: 2px` ergibt im Default-Box-Modell ein Element mit 244px tatsächlicher Breite — und zerschießt jedes Raster-Layout. Deshalb beginnt fast jedes moderne Stylesheet mit `* { box-sizing: border-box; }` als Reset. Dann entspricht `width: 200px` der tatsächlichen Breite, und Padding und Border werden innen abgezogen. Das ist eine der wenigen Stellen, an denen CSS einen schlechten Default hat — historisch gewachsen und nicht mehr änderbar, weil es unzählige bestehende Webseiten brechen würde.

Layoutmodelle im Überblick

Nicht jedes CSS-Layoutproblem ist dasselbe. Die wichtigsten Modelle beantworten unterschiedliche Fragen:

Normal Flow

Default-Verhalten von HTML:

- Block-Elemente stapeln sich untereinander
- Inline-Elemente fließen im Text
- Ausgangspunkt für jedes Layout

Flexbox

Verteilt Elemente entlang **einer Achse**:

- horizontal oder vertikal
- mit Abstand, Reihenfolge und Ausrichtung
- ideal für Reihen, Menüs, Toolbars

Grid

Organisiert Elemente in **Zeilen und Spalten**:

- zweidimensional
- explizite Rasterstruktur
- ideal für Seitenlayout, Dashboards, Galerien

Faustregel

- **Flow**: Was passiert ohne Layoutanweisung?
- **Flexbox**: Wie verteilen wir Inhalt entlang einer Achse?
- **Grid**: Wie definieren wir ein Raster?

Speaker notes

Flow, Flexbox und Grid sind drei verschiedene Layoutabstraktionen. Normal Flow ist der Ausgangszustand des Dokuments und folgt der Reihenfolge der Elemente. Flexbox löst Verteilungsprobleme entlang einer Dimension, etwa Navigationsleisten oder Kartenreihen. Grid löst Strukturprobleme in zwei Dimensionen, etwa Seitenlayouts mit Header, Sidebar, Inhalt und Footer. Viele CSS-Probleme entstehen, wenn ein Layoutmodell für eine Aufgabe genutzt wird, für die ein anderes Modell besser passt.

Normal Flow: Das Default-Layout des Browsers

Bevor wir Flexbox oder Grid einsetzen, sollten wir verstehen, was HTML **ohne besondere Layoutregeln** tut:

Block-Elemente beginnen typischerweise in einer neuen Zeile und nehmen die verfügbare Breite ein.

Inline-Elemente bleiben im laufenden Text und belegen nur so viel Platz wie ihr Inhalt braucht.

```
<p>Absatz <span>Inline-Text</span> nach dem Wort.</p>
<div>Dieses Block-Element steht darunter.</div>
```

Absatz Inline-Text nach dem Wort.

Dieses Block-Element steht darunter.

```
span {
  display: block;
}

div {
  display: inline;
}
```

Wichtig

HTML-Elemente haben Default-Displays. CSS kann diese Darstellung ändern, ohne die Semantik des Elements zu ändern.

Flow ist deshalb wichtig, weil Flexbox und Grid immer bewusst vom Normalfall abweichen.

Speaker notes

Normal Flow ist kein primitives Altverhalten, sondern die Basis des Webs. Dokumente, Artikel, Listen, Formulare und Absätze funktionieren bereits sinnvoll, bevor irgendein Layoutsystem aktiviert wird. Semantisches HTML erzeugt dadurch eine robuste Grundstruktur, die CSS später gezielt gestalten kann. Viele CSS-Probleme sind eigentlich Missverständnisse des Normal Flow: Ein `div` beginnt als Blockelement unter dem vorherigen Inhalt, während Inline-Elemente im Textfluss bleiben und keine eigene Blockhöhe erzeugen. Wichtig ist die Trennung von Semantik und Darstellung: Ein `span` bleibt semantisch ein Inline-Textcontainer, auch wenn CSS ihn mit `display: block` visuell wie einen Block darstellt.

Flexbox: Verteilung entlang einer Achse

Flexbox ist das richtige Werkzeug, wenn Elemente in **einer Zeile oder Spalte** organisiert werden sollen:

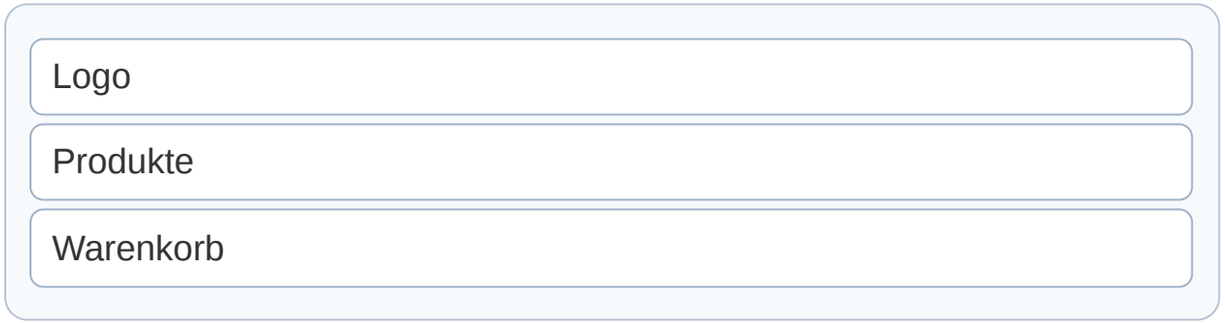
```css
.nav {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 1rem;
}
```

`display: flex` steht auf dem **übergeordneten Element** `.nav`. Die direkten Kinder werden dadurch zu Flex-Items.

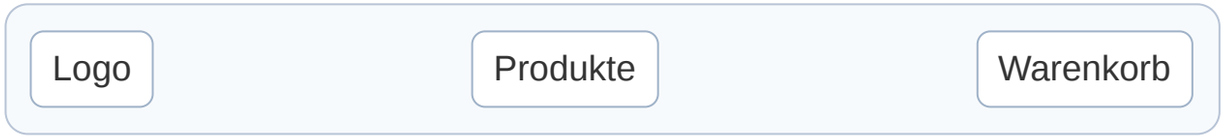
Typische Fragen:

- nebeneinander oder untereinander?
- wie viel Abstand?
- links, mittig oder rechts ausrichten?

Ohne Flex



Mit Flex



Ohne `display: flex` stapeln sich die Block-Elemente im Normal Flow. Mit `display: flex` verteilt der Container seine direkten Kinder entlang einer Achse.

Speaker notes

Flexbox ist ein eindimensionales Layoutmodell. Die Hauptachse wird durch `flex-direction` festgelegt, die Querachse ergibt sich daraus. `justify-content` arbeitet auf der Hauptachse, `align-items` auf der Querachse. Entscheidend ist, dass `display: flex` am Container steht: Nicht die einzelnen Links „werden flex", sondern der übergeordnete Navigationscontainer verteilt seine direkten Kinder. Genau diese Achsenlogik macht Flexbox für Menüs, Toolbars und Kartenreihen stark. Es ist aber das falsche Modell, wenn die Beziehung zwischen Zeilen und Spalten explizit beschrieben werden soll. Dann zwingt man Flexbox in Probleme hinein, die Grid klarer löst.

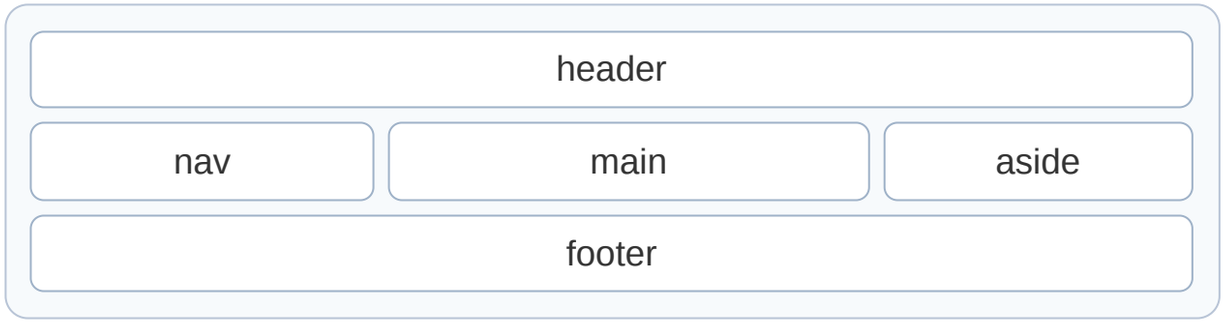
Grid: Ein explizites 2D-Raster

Grid ist sinnvoll, wenn das **Layout selbst** die Struktur vorgibt.
Explizit bedeutet hier: In CSS werden **Spalten**, **Zeilen** und oft auch die **Platzierung der Bereiche** vorgegeben.

```css
.shop-layout {
  display: grid;
  grid-template-columns: 240px 1fr 220px;
  grid-template-rows: auto 1fr auto;
  grid-template-areas:
    "header header header"
    "nav    main   aside"
    "footer footer footer";
  gap: 1rem;
}
```

`grid-template-columns` definiert die **Spalten**.
`grid-template-rows` definiert die **Zeilen**.
`grid-template-areas` ordnet benannte Bereiche im Raster an.

`1fr` bedeutet: nimm **einen Anteil des verbleibenden freien Platzes**.
`"header header header"` bedeutet: Der Bereich `header` belegt in dieser Zeile **alle drei Spalten**.



Bei Grid entsteht zuerst das Raster. Danach werden die Elemente in dieses Raster eingeordnet.

Speaker notes

Ein explizites 2D-Raster entsteht, wenn CSS zunächst die **Tracks** des Grids beschreibt, also Spalten und Zeilen. Danach werden Elemente auf diese Struktur abgebildet, entweder automatisch oder explizit über `grid-area`, `grid-column` und `grid-row`. Das unterscheidet Grid von Flexbox: Bei Flexbox entsteht die Anordnung aus der Reihenfolge der Items entlang einer Achse; bei Grid wird zuerst die räumliche Struktur definiert und dann belegt. `1fr` bedeutet „ein Anteil des noch freien Platzes“. In `240px 1fr 220px` sind die äußeren Spalten fest, die mittlere Spalte bekommt den restlichen Raum.

Responsives Grid ohne starre Breakpoints

Responsive Grid ist **weiterhin CSS Grid**. Jedoch ist die Anzahl der Spalten nicht fest vorgegeben:

Fixes Grid

```css
.product-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 1rem;
}
```

Immer drei Spalten — auch wenn der Platz knapp wird.

Responsives Grid

```css
.product-grid {
  display: grid;
  gap: 1rem;
  grid-template-columns:
    repeat(auto-fit, minmax(220px, 1fr));
}
```

So viele Spalten wie passen — sonst Umbruch.

`minmax(220px, 1fr)` bedeutet:

- nie schmaler als `220px`
- sonst flexibel wachsen

`auto-fit` erzeugt so viele Spalten wie möglich.
`minmax()` begrenzt jede Spalte zwischen Minimum und flexiblem Wachstum.

Breiter Container

Produkt A | Produkt B | Produkt C

Schmaler Container

Produkt A | Produkt B

Produkt C

Der Browser berechnet die Spaltenanzahl aus verfügbarem Platz und Mindestbreite. Keine eigene Media

Speaker notes

`auto-fit` und `minmax()` sind das Kernmuster moderner responsiver Kartenraster. Es handelt sich nicht um ein anderes Layoutmodell als Grid, sondern um eine andere Art, Tracks zu definieren. Bei `repeat(3, 1fr)` ist die Spaltenanzahl fix. Bei `repeat(auto-fit, minmax(220px, 1fr))` berechnet der Browser selbst, wie viele Tracks bei gegebener Breite sinnvoll sind. Das zeigt die deklarative Stärke von CSS: Man beschreibt Grenzen und Flexibilität, nicht einzelne Pixelkoordinaten. Im Unterschied zu klassischen Breakpoint-Layouts wird nicht festgelegt, dass bei einer bestimmten Breite genau zwei oder vier Spalten entstehen; stattdessen löst der Browser eine allgemeine Regel kontinuierlich auf.

Beispiel: Produktdetailseite mit Bild und Text

Beispiel-Code

```css
.product {
  display: grid;
  grid-template-areas: "image info";
  grid-template-columns: 1fr 1.2fr;
  gap: 2rem;
}

@media (max-width: 700px) {
  .product {
    grid-template-areas: "image" "info";
    grid-template-columns: 1fr;
  }
}
```

Warum hier Grid?

- Desktop beschreibt Bereiche als `"image info"`
- Mobile beschreibt Bereiche als `"image"` / `"info"`
- Layout bleibt als Struktur sichtbar
- Abfragbar mittesl CSS Media Queries

Flexbox wäre für eine einfache Zeile ebenfalls möglich. Grid ist hier klarer, weil Desktop- und Mobile-Layout als benannte Bereiche formuliert werden.

Situation: Links steht das Produktbild, rechts Beschreibung und Kaufbutton. Auf dem Smartphone soll das Bild oben und der Text darunter erscheinen.



Mobile: Bereiche untereinander




```
"image" / "info"
```

Speaker notes

Flexbox eignet sich gut, wenn Bild und Text einfach nebeneinander oder untereinander stehen sollen. Grid wird stärker, sobald das Layout selbst explizit modelliert wird, etwa mit benannten Bereichen, stabilen Relationen oder mehreren Umbrüchen. Im Beispiel ist Grid gut geeignet, weil Desktop und Mobile als zwei Layoutzustände desselben Bereichs lesbar bleiben: `"image info"` auf Desktop, `"image" "info"` auf Mobile. Die technische Entscheidung hängt also nicht am Alter des Layoutmodells, sondern daran, ob eine lineare Verteilung oder eine zweidimensionale Struktur ausgedrückt werden soll.

Welche Wahl ist hier besser?

Flexbox passt, wenn

- nur eine Hauptachse wichtig ist
- die Elemente einfach verteilt werden
- Reihenfolge und Struktur weitgehend linear bleiben

Grid passt, wenn

- Zeilen und Spalten gemeinsam wichtig sind
- Bereiche explizit benannt werden sollen
- responsives Umschalten strukturell klar bleiben soll

Faustregel: Flexbox, wenn der Inhalt verteilt wird. Grid, wenn das Layout als Raster vorgegeben wird.

Speaker notes

In der Praxis arbeiten beide Modelle zusammen. Grid organisiert häufig die grobe Seitenstruktur, Flexbox den Inhalt einzelner Bereiche wie Header, Kartenkopf oder Button-Leiste. Das ist kein Widerspruch, sondern die normale Architektur moderner Oberflächen. Wer versucht, alles nur mit Flexbox oder alles nur mit Grid zu lösen, verliert Ausdrucksstärke und Lesbarkeit. Gute CSS-Architektur beginnt daher mit der Frage: Welche räumliche Beziehung will ich eigentlich ausdrücken?

Takeaway

CSS existiert als eigene Sprache, weil dasselbe HTML in **verschiedenen Kontexten** verschieden dargestellt werden muss — Screen, Print, Mobile, Dark Mode. CSS ist **deklarativ**: Man beschreibt Constraints, der Browser berechnet Pixel. Spezifität bestimmt, welche Regel gewinnt. Flexbox und Grid lösen unterschiedliche Verteilungsprobleme (Inhalt verteilen vs. Raster vorgeben). Responsive Design mit Mobile-First ist Voraussetzung. Die wichtigste Grenzfrage: **Kann CSS das allein** (Hover, Transition, Media Query), oder braucht es JavaScript? Nur wenn neuer Zustand erzeugt werden muss, ist JS die richtige Schicht.

Speaker notes

Deklarativität ist das definierende Merkmal von CSS: Der Entwickler beschreibt Constraints (Breite, Abstände, Farbe, Layout-Regeln), der Browser berechnet die konkreten Pixel — abhängig von Viewport, Schriftgröße, Gerätepixeldichte und verfügbarem Raum. Das ist fundamental anders als Canvas-Rendering oder SVG-Zeichnen mit Pixelkoordinaten. Die Deklarativität ist der Grund, warum `grid-template-columns: repeat(auto-fit, minmax(220px, 1fr))` automatisch auf 300px-Screens mit einer Spalte und auf 1400px-Screens mit sechs Spalten reagiert — ohne Media Query. Imperative CSS-Ansätze (z.B. Breiten per JavaScript explizit setzen) kämpfen gegen dieses Modell und erzeugen Layout-Bugs bei Reflows und Resize-Events.

Web APIs — die Schnittstellen des Browsers

Leitfrage: Was sind Web APIs — und warum gehören sie nicht zur JavaScript-Sprache selbst, sondern zum Browser?

Speaker notes

Die JavaScript-Sprache (ECMAScript) ist erstaunlich klein: Variablen, Funktionen, Schleifen, Objekte, aber keine Datei-, Netzwerk- oder DOM-Operationen. Alles, was JavaScript im Browser praktisch tun kann, kommt von **Web APIs**, also Schnittstellen, die der Browser bereitstellt. Derselbe JavaScript-Code hat in unterschiedlichen Umgebungen unterschiedliche APIs: im Browser `window` und `fetch`, in Node.js `process` und `fs`, in einem Service Worker keinen direkten DOM-Zugriff.



Web APIs: Definition und Überblick

Web APIs sind Browser-Schnittstellen, die JavaScript zusätzliche Fähigkeiten geben: den DOM verändern, Netzwerke ansprechen, Daten speichern, Gerätefunktionen nutzen oder Medien steuern.

Typische Browser-APIs

- DOM API
- Fetch API
- Web Storage API
- Geolocation API
- Canvas / WebGL
- Web Workers

Warum das wichtig ist

- JavaScript allein beschreibt die Sprache
- Web APIs machen sie im Browser praktisch nutzbar
- viele Web-Funktionen sind erst durch diese APIs möglich

> **Beispiel:** `fetch()` lädt Daten, `localStorage` speichert Zustand, DOM APIs aktualisieren die Oberfläche.

Speaker notes

Der Begriff „API" ist im Web-Kontext doppeldeutig: Er kann eine **Web API** im Sinne einer Browser-Schnittstelle meinen (was dieses Modul behandelt) oder eine **HTTP-API** im Sinne eines serverseitigen Endpunkts (was Vorlesung 04 behandelt). Beide sind APIs im breiten Sinn „Application Programming Interface". Wenn Sie in diesem Modul von APIs sprechen, meinen Sie Schnittstellen *des Browsers* an den JavaScript-Code. Diese kommen nicht aus ECMAScript selbst — JavaScript als Sprache kennt weder `window`, noch `document`, noch `fetch`. Es sind Erweiterungen, die der Browser zur Laufzeit bereitstellt.

Wofür nutzt man sie?

Aufgabe	Beispiel-API
HTML oder CSS zur Laufzeit ändern	DOM API
Daten vom Server laden	Fetch API
Zustand im Browser speichern	Web Storage API
Standorte abfragen	Geolocation API
Lange Berechnungen auslagern	Web Workers

Kernaussage: Web APIs sind die Schnittstelle zwischen JavaScript und den Fähigkeiten des Browsers.

> **Grenze:** Nicht jede API ist Teil von JavaScript selbst. Viele Funktionen kommen vom Browser, nicht von der Sprache.

Speaker notes

Die Unterscheidung ist praktisch relevant: Derselbe JavaScript-Code läuft je nach Umgebung unterschiedlich. Im Browser funktioniert `localStorage`, in Node.js nicht. In Node.js funktioniert `fs.readFile`, im Browser nicht. Wer Isomorphic JavaScript (Code, der auf Server und Client läuft) schreibt, muss diese Unterschiede kennen — und muss manchmal mit Polyfills oder Feature-Detection (`if (typeof window !== 'undefined')`) arbeiten. Moderne Full-Stack-Frameworks wie Next.js abstrahieren diese Trennung teilweise, aber unter der Haube bleibt sie bestehen.

Fetch API: Die Client-Seite von HTTP

Die Fetch API ist die Brücke zwischen Frontend (Vorlesung 03) und Protokoll (Vorlesung 04):

```
// GET: Produktliste laden
const response = await fetch('/api/products?category=electronics&sort=-price');
const products = await response.json();

// POST: Bestellung aufgeben
const result = await fetch('/api/orders', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ productId: 42, quantity: 1 })
});
if (result.status === 201) {
  const order = await result.json();
  window.location.href = `/orders/${order.id}`;  // Weiterleitung
}
```

Fetch-Konzept	HTTP-Gegenstück
`method: 'POST'`	HTTP-Methode — definiert die Absicht
`headers`	HTTP Request Headers — Metadaten
`response.status`	HTTP-Statuscode — Ergebnis der Operation
`response.json()`	Body parsen — die Repräsentation der Ressource

> `fetch()` ist die JavaScript-Abstraktion, aber die Konzepte darunter sind HTTP: Methode, Header, Statuscode und Body.

Speaker notes

Die Fetch API ist die moderne Schnittstelle zwischen JavaScript und HTTP und hat den älteren `XMLHttpRequest` (XHR) abgelöst. `fetch()` gibt ein **Promise** zurück, das sich auflöst, sobald die *Header* der Antwort eingetroffen sind, nicht erst nach dem vollständigen Body. `response.json()` ist ein zweiter asynchroner Schritt, der den Body parst. Diese Trennung ermöglicht Streaming und erlaubt es, Status oder `Content-Type` zu prüfen, bevor große Antwortkörper vollständig verarbeitet werden.

Was kann beim Fetch schiefgehen?

Aufgabe: Dieser Code funktioniert im Normalfall. Welche Probleme können auftreten — und wie sollte man sie behandeln?

```
const response = await fetch('/api/products/42');
const product = await response.json();
document.querySelector('.name').textContent = product.name;
document.querySelector('.price').textContent = product.price + ' €';
```

```
button.addEventListener('click', async () => {
  try {
    const response = await fetch('/api/products/42');
    if (!response.ok) throw new Error(`HTTP ${response.status}`);

    const product = await response.json();
    renderProduct(product);
  } catch (err) {
    showError('Produkt konnte nicht geladen werden.');
  }
});
```

Das ist ereignisgesteuert **und** Promise-basiert: Der Klick startet den Ablauf, `await` wartet auf die asynchrone Fetch-Fortsetzung.

Speaker notes

Die fünf Fehlerfälle dieses Code-Snippets decken das gesamte Spektrum netzwerkbasierter Fehlerquellen ab: (1) **Netzwerk nicht erreichbar** — `fetch` wirft eine Exception, kein Response-Objekt entsteht. (2) **HTTP-Fehler (4xx/5xx)** — `fetch` liefert ein Response-Objekt mit `response.ok === false`; `.json()` parst den Fehler-Body, der häufig HTML statt JSON enthält. (3) **Langsame Verbindung** — UI zeigt veraltete oder keine Daten, kein Feedback für den Nutzer. (4) **Kein JSON im Body** — `.json()` wirft eine SyntaxError-Exception. (5) **Fehlendes DOM-Element** — `querySelector` gibt `null` zurück, Zugriff auf `.textContent` löst TypeError aus. Ereignisgesteuert bedeutet hier: Ein Nutzerereignis startet die asynchrone Operation. `async/await` ersetzt nicht das Event-Modell, sondern strukturiert den Promise-basierten Ablauf innerhalb des Event-Handlers. Eine subtile Falle bei Fetch: `fetch()` wirft *nur* dann eine Exception, wenn das Netzwerk komplett versagt, etwa bei DNS-Fehlern oder fehlender Verbindung. HTTP-Statuscodes wie 404 oder 500 gelten technisch als erfolgreich übertragene Antworten; `fetch` liefert ein gültiges Response-Objekt, und `response.ok` ist `false`. Ohne Statusprüfung versucht Code häufig, HTML-Fehlerseiten oder leere Bodies als JSON zu parsen. Robuste Patterns prüfen deshalb `response.ok` und den `Content-Type`, bevor `response.json()` aufgerufen wird.

Was kann beim Fetch schiefgehen?

Aufgabe: Dieser Code funktioniert im Normalfall. Welche Probleme können auftreten — und wie sollte man sie behandeln?

```
const response = await fetch('/api/products/42');
const product = await response.json();
document.querySelector('.name').textContent = product.name;
document.querySelector('.price').textContent = product.price + ' €';
```

```
button.addEventListener('click', async () => {
  try {
    const response = await fetch('/api/products/42');
    if (!response.ok) throw new Error(`HTTP ${response.status}`);

    const product = await response.json();
    renderProduct(product);
  } catch (err) {
    showError('Produkt konnte nicht geladen werden.');
  }
});
```

Problem	Was passiert	Lösung
Server nicht erreichbar	`fetch` wirft Exception	`try/catch` + Fehlermeldung anzeigen
404 Not Found	`response.ok` ist `false`, `.json()` liefert Fehler-Body	Status prüfen vor `.json()`
Langsame Verbindung	UI zeigt alte/keine Daten	Loading-Indicator anzeigen
Response ist kein JSON	`.json()` wirft Exception	`Content-Type` prüfen
DOM-Element existiert nicht	`querySelector` liefert `null` → Fehler	Null-Check oder optionale Verkettung (`?.`)

Robuste Fehlerbehandlung ist keine Kür — sie ist der Unterschied zwischen einem Prototyp und einem Produktivsystem.

Das ist ereignisgesteuert **und** Promise-basiert: Der Klick startet den Ablauf, `await` wartet auf die asynchrone Fetch-Fortsetzung.

Promises und asynchrone Verarbeitung

Ein **Promise** repräsentiert das *zukünftige Ergebnis* einer asynchronen Operation. Es kennt drei Zustände: `pending` → `fulfilled` oder `rejected`.

Klassischer Promise-Stil mit Callback-Verkettung

```
fetch('/api/products/42')                  // gibt Promise zurück
  .then(response => {                      // läuft, wenn Header da sind
    if (!response.ok) throw new Error(response.status);
    return response.json();                // gibt wieder ein Promise zurück
  })
  .then(product => renderProduct(product)) // läuft, wenn JSON geparst ist
  .catch(err => showError(err.message));   // fängt Fehler aus jeder Stufe
```

Konzept	Bedeutung
`await`	Pausiert die `async`-Funktion, bis das Promise *settled* ist — blockiert **nicht** den Main Thread
`Promise.all([p1, p2])`	Wartet parallel auf mehrere Promises; scheitert, sobald *eines* rejected
`Promise.allSettled([...])`	Wartet auf alle, gibt Erfolg *und* Fehler zurück
`try / catch` um `await`	Fängt sowohl Netzwerk- als auch Logikfehler — ersetzt `.catch()`

`async / await` — derselbe Ablauf, linearer Code

```
async function loadProduct() {
  try {
    const product = await productApi.fetch(42);
    renderProduct(product);
  } catch (err) {
    showError(err.message);
  }
}
```

> `productApi.fetch(...)` implementiert dieselbe Datenlade-Methode wie die `.then(...)`-Kette: erst `fetch`, dann `response.json()`, dann `return product`. Der UI-Code ruft diese Methode nur noch mit `await` auf.

```
const productApi = {
  async fetch(id) {
    // entspricht: fetch(...).then(response => ...)
    const responsePromise = fetch(`/api/products/${id}`);
    const response = await responsePromise;

    if (!response.ok) throw new Error(response.status);

    // entspricht: response.json().then(product => ...)
    const productPromise = response.json();
    const product = await productPromise;

    return product;
  }
};
```

Speaker notes

Promises wurden mit ES2015 (ES6) standardisiert und lösten das Callback-Hell-Problem von XHR und Node.js-Style-Callbacks. Ein Promise ist ein Objekt, das eine Operation kapselt, deren Ergebnis noch nicht vorliegt. Der entscheidende Vorteil gegenüber Callbacks: Promises sind *komponierbar* (`.then` verkettet, `Promise.all` parallelisiert) und haben einheitliche Fehlerbehandlung (`.catch` fängt Fehler aus jeder Stelle der Kette). `async / await` (ES2017) ist rein syntaktisch — der Compiler übersetzt es in eine Promise-Kette mit Generatoren. Wichtig für das mentale Modell: `await` blockiert *nur* die umgebende `async`-Funktion, der Event Loop läuft weiter und kann andere Events bearbeiten. Genau deshalb friert die UI nicht ein, obwohl Code wie sequentielle Synchron-Logik aussieht. Ein häufiger Fehler: zwei unabhängige `await`-Aufrufe sequenziell statt parallel mit `Promise.all` ausführen — das verschwendet wartbare Zeit (zwei Netzwerk-Roundtrips nacheinander statt gleichzeitig).



await im Zeitverlauf: zwei Fetch-Requests



Speaker notes

Wichtigster Punkt der Skizze: Die grünen Blöcke auf dem Main Thread zeigen, dass während `await` *andere* Browser-Events ungehindert laufen — Klicks, Rendering, Animationen. Genau das ist der Vorteil von `async/await` gegenüber synchronem I/O: Bei einem hypothetischen synchronen Fetch wäre der Main Thread während der gesamten Netzwerkdauer blockiert, die Seite würde einfrieren („page frozen"). `await` gibt den Main Thread während des Wartens frei, sodass die Event Queue weiter abgearbeitet werden kann. Die *eigene* `async`-Funktion pausiert (orange Blöcke nur am Anfang/Ende), aber der Main Thread tut es nicht. Der zweite Vorteil — sichtbar im unteren Block — ist *Parallelität auf der Netzwerk-Ebene*: `Promise.all([fetch(A), fetch(B)])` übergibt beide Requests gleichzeitig an die Browser-API, sodass nur eine einzige Wartephase entsteht statt zwei. Faustregel: Zwei aufeinanderfolgende `await`s sind nur dann korrekt, wenn der zweite Request vom Ergebnis des ersten *abhängt*; ansonsten gehört das Aufrufpaar in ein `Promise.all`.

Cookies: kleine Schlüssel-Werte, automatisch zum Server

Cookies sind kleine, vom Browser gespeicherte Schlüssel-Wert-Paare. Im Unterschied zu allen anderen Storage-APIs werden sie bei **jedem** HTTP-Request an die zugehörige Domain *automatisch* mitgeschickt — das macht sie zur klassischen Wahl für Sessions und Authentifizierung.

Setzen — Server (üblich)

```
HTTP/1.1 200 OK
Set-Cookie: session=abc123; Max-Age=3600;
          HttpOnly; Secure; SameSite=Strict
```

Setzen — Client (selten)

```
document.cookie = 'theme=dark; max-age=31536000; SameSite=Lax';
```

Lesen — Client

```
// alle (nicht-HttpOnly) Cookies als ein String
const all = document.cookie;
// → "theme=dark; locale=de-AT"
```

Wichtige Attribute

Attribut	Wirkung
`HttpOnly`	Für JavaScript **unsichtbar** — schützt vor XSS-Diebstahl
`Secure`	Nur über HTTPS übertragen
`SameSite=Strict|Lax|None`	Sendet Cookie nicht (oder nur Top-Level) bei Cross-Site-Requests — schützt vor CSRF
`Max-Age` / `Expires`	Lebensdauer; ohne beides → Session-Cookie
`Domain` / `Path`	Auf welche Hosts/URLs der Cookie geht

Typische Anwendungen

- Session-ID für angemeldete Nutzer
- CSRF-Token, Sprache, Theme
- Tracking / Analytics (Drittanbieter-Cookies)

Faustregel: Auth-Tokens gehören in einen Cookie mit `HttpOnly + Secure + SameSite=Strict`. UI-State

Speaker notes

Cookies sind die älteste clientseitige Speichertechnologie des Webs (Netscape, 1994) und der Grund, warum HTTP überhaupt zustandsbehaftete Sessions ermöglichen kann — das Protokoll selbst ist zustandslos. Die wichtigste Eigenschaft, die Cookies von `localStorage` unterscheidet: Sie werden vom Browser bei jedem passenden Request *automatisch* in den `Cookie`-Header gepackt und mitgeschickt. Der Server muss sie also nicht aktiv anfragen — sie sind einfach da. Das ist Segen und Fluch zugleich: praktisch für Sessions, gefährlich bei Cross-Site-Angriffen (CSRF). Genau deshalb ist `SameSite` heute essenziell — moderne Browser setzen `SameSite=Lax` als Default. `HttpOnly` schließt JavaScript komplett vom Lesen aus; das bedeutet, ein erfolgreicher XSS-Angriff kann das Auth-Cookie *nicht* exfiltrieren. Der Tradeoff: HttpOnly-Cookies können nicht von SPA-Code gelesen werden — Single-Page-Apps verwenden deshalb oft eine Kombination aus HttpOnly-Refresh-Cookie und kurzlebigem Access-Token im Memory. Drittanbieter-Cookies (Cookies, die zu einer fremden Domain gehören und in iframes oder beim Laden von Drittanbieter-Skripten gesetzt werden) sind die Grundlage für Cross-Site-Tracking und werden von allen modernen Browsern stark eingeschränkt oder ganz blockiert (Safari ITP, Firefox ETP, Chrome Privacy Sandbox).



Web Storage: localStorage, sessionStorage, Cookies

Browser-Speicher unterscheidet sich vor allem danach, **wie lange** Daten leben und **wer** sie lesen oder automatisch mitsenden kann.

API	Lebensdauer	Größe	Zugriff
`localStorage`	Persistent (bis manuell gelöscht)	~5–10 MB	Nur Client (JS)
`sessionStorage`	Nur aktuelle Browser-Session	~5–10 MB	Nur Client (JS)
`Cookies`	Konfigurierbar (Session oder Ablaufdatum)	~4 KB	Client + **automatisch an Server**
`IndexedDB`	Persistent	Gigabyte-Bereich	Client (asynchron)

```
// localStorage: einfache Key-Value-Speicherung
localStorage.setItem('cart', JSON.stringify(cartItems));
const cart = JSON.parse(localStorage.getItem('cart') || '[]');

// sessionStorage: nur für die aktuelle Tab-Session
sessionStorage.setItem('wizard-step', '3');

// Cookies: werden bei jedem HTTP-Request mitgesendet
document.cookie = 'theme=dark; max-age=31536000; SameSite=Strict';
```

Wann was?

- **Cookies** für alles, was der **Server wissen muss** (Session-ID, Authentifizierung)
- **localStorage** für Client-State, der **über Sessions** erhalten bleiben soll (Warenkorb, UI-Einstellungen)
- **sessionStorage** für temporären State, der **nur im aktuellen Tab** gilt (Wizard-Fortschritt)
- **IndexedDB** für große Datenmengen oder Offline-Anwendungen

Speaker notes

Ein häufiger Fehler: Authentifizierungs-Tokens in `localStorage` zu speichern. Das ist verlockend, weil es einfach ist, aber gefährlich: Jeder JavaScript-Code auf der Seite (inklusive eingebettete Drittanbieter-Skripte und XSS-Angriffe) kann `localStorage` lesen. Cookies mit `HttpOnly` und `SameSite=Strict` sind sicherer, weil sie JavaScript verborgen bleiben und nur vom Server gelesen werden können. Das ist eine der Architektur-Entscheidungen, die Vorlesung 7 (Sicherheit) vertieft. Für Warenkorb und UI-State ist `localStorage` dagegen völlig in Ordnung — kein Angreifer gewinnt etwas durch Kenntnis des Warenkorbinhalts.

History API: Navigation ohne Seiten-Reload

Single-Page Applications navigieren ohne vollständigen Seiten-Reload. Der Browser-Zurück-Button muss trotzdem funktionieren. Die **History API** macht das möglich:

```
// URL ändern, ohne neu zu laden
history.pushState({ page: 'products' }, '', '/products');

// Neue Inhalte fetch'en und rendern
const data = await fetch('/api/products').then(r => r.json());
renderProducts(data);

// Zurück-Button abfangen
window.addEventListener('popstate', (event) => {
  if (event.state?.page === 'products') renderProducts(/* ... */);
});
```

Methode / Event	Zweck
`history.pushState(state, '', url)`	URL ändern, Eintrag zum Verlauf hinzufügen
`history.replaceState(...)`	URL ändern, **ohne** neuen Eintrag
`popstate` Event	Browser-Zurück-/Vorwärts-Button

> Client-side Routing muss immer zwei Wege behandeln: aktive Navigation per `pushState` und passive Navigation über den Zurück-/Vorwärts-Button.

Speaker notes

Die History API ist der technische Kern jedes Client-Side-Routers — React Router, Vue Router und SvelteKit nutzen sie unter der Haube. Das Problem, das sie löst: Ohne History API würde jede Navigation in einer SPA die URL unverändert lassen, und der Browser-Zurück-Button würde die SPA verlassen. Mit `pushState` bleibt die URL konsistent mit dem UI-Zustand, und der Zurück-Button funktioniert wie erwartet. Wichtige Feinheit: `pushState` löst *kein* `popstate`-Event aus — das Event feuert nur beim Zurück-Navigieren. Eigene Navigationslogik muss also zwei Pfade unterstützen: aktives Navigieren (UI ruft `pushState` + Render auf) und passives Zurück-Navigieren (popstate-Event triggert Render).

Web Workers vs Service Workers

Beide laufen in einem **separaten Thread** außerhalb des Haupt-Event-Loops — aber mit unterschiedlichen Zwecken.

Aspekt	Web Worker	Service Worker
Zweck	CPU-intensive Berechnungen auslagern	Netzwerk-Proxy, Offline-Support, Push
Lebensdauer	Gebunden an Tab/Seite	Persistent, auch wenn Tab geschlossen
DOM-Zugriff	Nein	Nein
Fetch-Interception	Nein	Ja — fängt alle Netzwerk-Requests ab
Typischer Einsatz	Bildverarbeitung, Datenanalyse, Kryptographie	Progressive Web Apps, Caching, Push-Benachrichtigungen

```
// Web Worker: teure Berechnung auslagern
const worker = new Worker('compute.js');
worker.postMessage({ data: largeDataset });
worker.onmessage = (e) => displayResult(e.data);

// Service Worker: Fetch abfangen für Offline-Support
self.addEventListener('fetch', (event) => {
  event.respondWith(
    caches.match(event.request).then(cached => cached || fetch(event.request))
  );
});
```

> Web Worker entlasten die UI bei Rechenarbeit. Service Worker sitzen zwischen Anwendung und Netzwerk und machen Offline-/Cache-Strategien möglich.

Speaker notes

Service Workers sind das Fundament jeder Progressive Web App (PWA). Sie ermöglichen Offline-Betrieb, weil sie Netzwerk-Requests abfangen und aus einem Cache beantworten können, selbst wenn die Verbindung weg ist. Web Workers lösen ein anderes Problem: Lange Berechnungen wie Bildverarbeitung, Parsen großer JSON-Dateien oder Kryptographie blockieren sonst die UI. Im Worker läuft dieser Code in einem echten zweiten OS-Thread, ohne den Hauptthread zu beeinträchtigen. Beide Worker-Typen haben keinen DOM-Zugriff und kommunizieren mit dem Hauptthread über `postMessage`.

Takeaway

Web APIs sind die **Fähigkeiten**, die der Browser JavaScript zur Verfügung stellt — nicht Teil der Sprache selbst, sondern Erweiterungen. Die wichtigsten Kategorien sind DOM-Manipulation, Netzwerk (Fetch), Speicher (localStorage/Cookies), Hintergrund-Threads (Web/Service Workers) und History. Welche API wofür eingesetzt wird, ist eine Architekturfrage mit Konsequenzen für Sicherheit, Performance und Offline-Fähigkeit.

Speaker notes

Die Trennung zwischen ECMAScript (Sprache) und Web APIs (Plattform) hat konkrete technische Konsequenzen: Derselbe JavaScript-Code läuft in unterschiedlichen Umgebungen unterschiedlich — `localStorage` ist im Browser global verfügbar, in Node.js nicht definiert; `fs.readFile` ist in Node.js verfügbar, im Browser nicht. Polyfills implementieren eine Web API für Umgebungen, die sie nicht nativ unterstützen — z.B. `fetch` in älteren Node-Versionen. Feature-Detection (`if (typeof serviceWorker !== 'undefined')`) ist stabiler als Browser-Detection, weil sie direkt prüft ob die Fähigkeit vorhanden ist, unabhängig von Browser-Version oder User-Agent-String. Isomorphic JavaScript (Code, der auf Server und Client gleich läuft) muss diese Grenze mit Adapter-Schichten oder Conditional Imports explizit handhaben.

Moderne Web Frontends

Leitfrage: Nachdem HTML, CSS und JavaScript einzeln geklärt sind: Welche Performance- und Architekturfolgen entstehen aus ihrem Zusammenspiel im Browser?

Speaker notes

Die gemeinsame Rendering-Pipeline des Browsers besteht aus sechs sequenziellen Phasen: (1) HTML parsen → DOM aufbauen, (2) CSS parsen → CSSOM aufbauen, (3) DOM + CSSOM → Render Tree (unsichtbare Elemente werden ausgeblendet), (4) Layout — Geometrie jedes Render-Tree-Knotens berechnen, (5) Paint — Pixel zeichnen, (6) Composite — Layer zusammenstellen. JavaScript kann in jeder Phase eingreifen: DOM-Mutationen invalidieren den Render Tree ab Schritt 3, `element.offsetHeight` erzwingt Layout synchron, CSS-Property-Änderungen mit `transform` können Schritte 4–5 überspringen. Performance-Probleme entstehen fast immer an diesen Schnittstellen, nicht innerhalb einer einzelnen Technologie.

Warum Frameworks existieren: Das Skalierungsproblem

Unser Warenkorb-Button funktioniert mit `querySelector` und `addEventListener`. Was passiert, wenn die Anwendung wächst?

```
button.addEventListener('click', () => {
  badge.textContent = '3';
});
```

Imperativ: DOM nachführen

- Elemente suchen
- Events registrieren
- DOM manuell aktualisieren
- Zustand an mehreren Stellen synchron halten

Deklarativ: UI aus Zustand

- Zustand beschreiben
- Komponenten rendern
- Framework berechnet DOM-Änderungen
- UI bleibt konsistent zu `state`

Frameworks lösen kein Syntax-Problem, sondern ein **Zustandsmanagement-Problem**: Wie bleibt die UI konsistent, wenn sich Daten ändern?

Speaker notes

Die zentrale Erkenntnis: Frameworks wie React, Vue, Angular oder Svelte sind keine syntaktischen Verschönerungen, sondern eine konzeptionelle Verschiebung. Imperatives DOM („greife Element, ändere Property") wird durch deklarative UI ersetzt („so soll es aussehen, wenn der Zustand X ist"). Das Framework berechnet anschließend die notwendigen DOM-Änderungen. Für kleine Anwendungen ist das Overhead; 50 Zeilen Vanilla JS reichen oft. Sobald aber Zustand an mehreren Stellen synchron gehalten werden muss — Warenkorb-Badge, Sidebar, Modal, Produktliste, Filter — wird imperatives DOM-Tracking unwartbar und der Framework-Overhead lohnt sich.

Wie hat sich das Zusammenspiel zwischen HTML/CSS/JS verändert?

Ära	Muster	Zusammenspiel
Klassisch (2000er)	Server rendert HTML, CSS als separate Datei, JS für kleine Effekte	Strikte Trennung: HTML-Datei + CSS-Datei + JS-Datei
jQuery-Ära (2006–2015)	HTML vom Server, JS manipuliert DOM direkt	DOM als zentrale Schnittstelle
Komponentenbasiert (ab 2013)	React/Vue/Angular: HTML + CSS + JS pro Komponente	Trennung nach **Verantwortung**, nicht nach Technologie
Server-First (ab 2020)	Next.js/SvelteKit: Server rendert, Client hydriert	Erst HTML vom Server, dann JavaScript macht es interaktiv

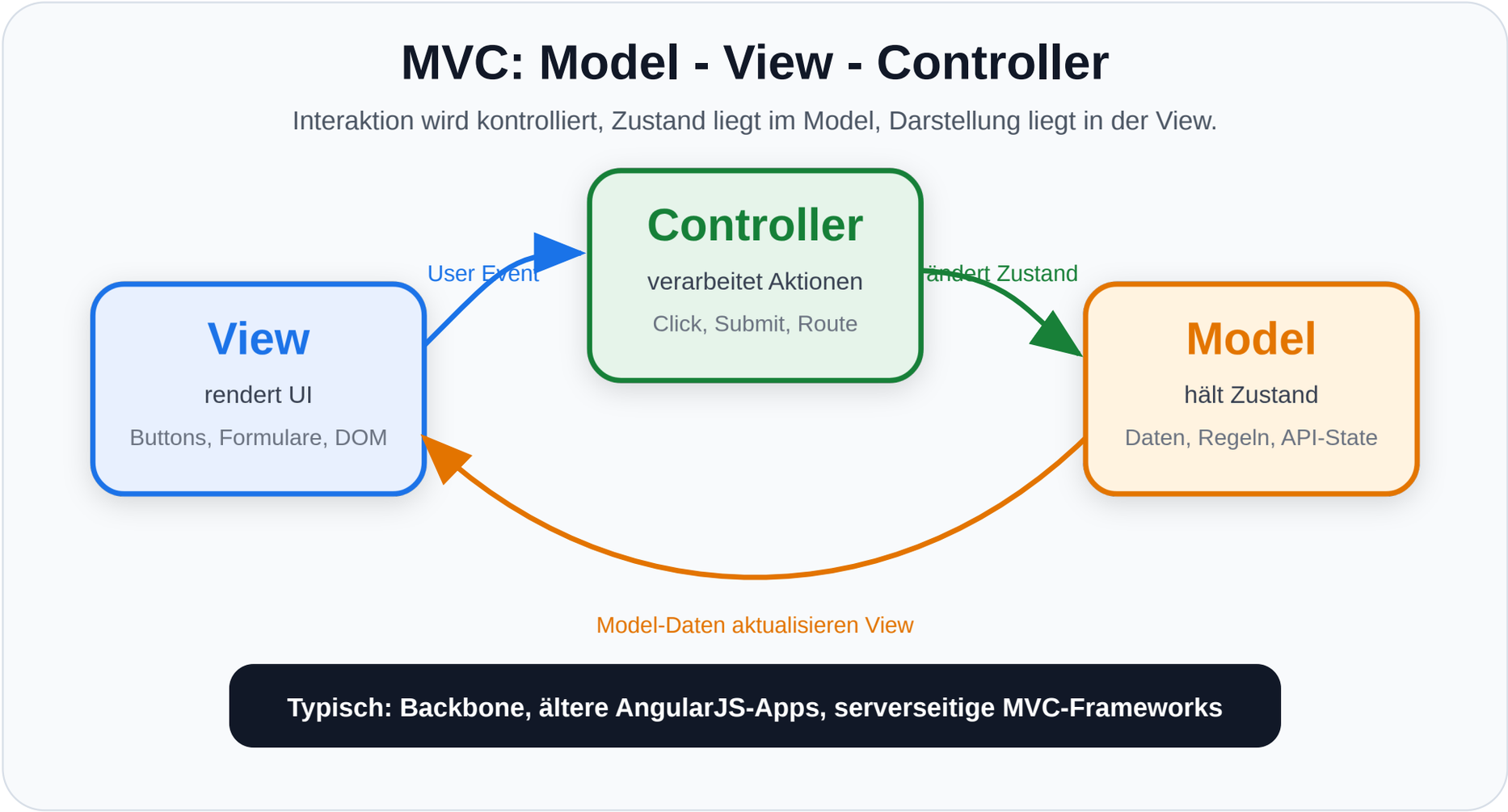
Client-hydriert bedeutet: Der Server liefert bereits fertiges HTML an den Browser. Danach lädt JavaScript nach und **bindet Verhalten an dieses vorhandene HTML** (Event-Listener werden registriert, Komponenten bekommen ihren interaktiven Zustand, die Seite wird nutzbar, ohne dass der Browser alles neu rendern muss).

> Welche Architekturen gibt es nun zur Auftrennung von Darstellung (View), Daten (Model) und Kontrolllogik (Control).

Speaker notes

Die vier Ären des Web-Frontends repräsentieren unterschiedliche Antworten auf dieselbe architektonische Frage: Wo gehört die Rendering-Logik hin? In den 2000ern blieb sie auf dem Server (HTML-Ausgabe). In der jQuery-Ära wanderte sie schrittweise in den Client, ohne formales Zustandsmodell. Mit React/Vue (ab 2013) entstand ein formales Client-Modell mit deklarativem Zustandsmanagement. Server-First-Frameworks (Next.js, SvelteKit, ab 2020) kehren zum Server zurück, aber mit Client-Hydration als Erweiterung. Die React-Komponentenidee hebt Separation of Concerns nicht auf — sie verschiebt sie von der Technologieachse (HTML/CSS/JS-Dateien) auf die Fachlichkeitsachse (Komponente: Produktkarte, Warenkorb, Navigation). Beide Interpretationen sind gültig und spiegeln unterschiedliche Systemkomplexitäten wider.

MVC: Model - View - Controller



MVC trennt Darstellung, Zustand und Eingabeverarbeitung. Die View zeigt Daten, der Controller verarbeitet Aktionen, das Model hält fachlichen Zustand.

Speaker notes

MVC stammt ursprünglich aus Smalltalk und wurde später für Web-Frameworks angepasst. Im klassischen Web liegt MVC oft serverseitig: Ein Controller nimmt HTTP-Requests entgegen, lädt oder verändert Model-Daten und wählt anschließend eine View, die HTML rendert. Im Frontend ist MVC weniger eindeutig, weil Browser-Views, DOM-Events und clientseitiger Zustand enger gekoppelt sind.

MVC am Mini-Beispiel

View

```
<input id="name">
<button id="save">Speichern</button>
<p id="output"></p>
```

Model

```
const model = { name: '' };
```

Controller

```
document.querySelector('#save')
  .addEventListener('click', () => {
    model.name = document.querySelector('#name').value;
    render();
  });

function render() {
  document.querySelector('#output').textContent =
    `Hallo ${model.name}`;
}
```

Vorteil: Sehr direkt: Event → Controller → Model → View.

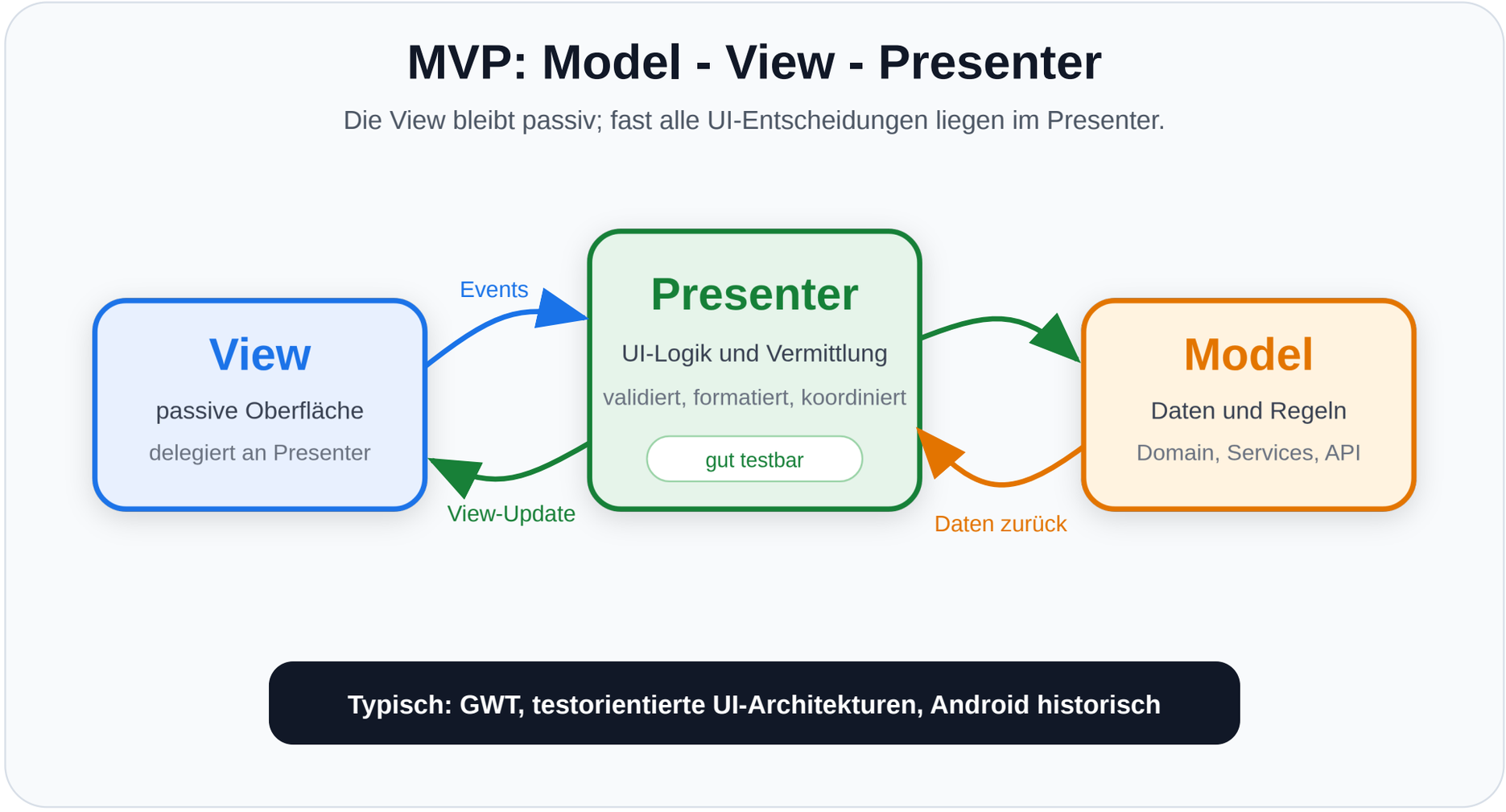
Nachteil: Schwerer isoliert testbar: Controller kennen DOM-Details, und View-Code kann eigene Logik enthalten.

MVC: Der Controller reagiert auf die Eingabe, ändert das Model und stößt das Rendering der View an.

Speaker notes

In diesem Beispiel ist die View der sichtbare DOM-Ausschnitt, das Model ist ein einfaches Zustandsobjekt, und der Controller ist der Event-Handler am Button. Die Kopplung ist bewusst direkt gehalten: Der Controller kennt sowohl das Eingabeelement als auch das Model und ruft anschließend `render()` auf. In größeren MVC-Systemen würde diese Rolle auf eigene Controller-Klassen oder Router-Handler verteilt.

MVP: Model - View - Presenter



MVP macht die View passiv. Der Presenter enthält die UI-Logik und spricht die View über ein Interface an.

Speaker notes

MVP wurde populär, weil es UI-Logik testbarer macht. Die View enthält möglichst wenig Entscheidungscode und delegiert Interaktionen an den Presenter. Der Presenter kann dadurch häufig ohne echte UI getestet werden, indem das View-Interface durch einen Mock ersetzt wird. Der Preis ist zusätzlicher Boilerplate-Code zwischen View und Presenter.

MVP am Mini-Beispiel

View: Oberfläche + Interface

```html
<input id="name">
<button id="save">Speichern</button>
<p id="output"></p>
```

```
const view = {
  getName: () =>
    document.querySelector('#name').value,
  showGreeting: (text) =>
    document.querySelector('#output').textContent = text
};
```

Model + Presenter

```
const model = { name: '' };

const presenter = {
  save() {
    model.name = view.getName();
    view.showGreeting(`Hallo ${model.name}`);
  }
};

document.querySelector('#save')
  .addEventListener('click', () => presenter.save());
```

Vorteil: UI-Logik ist gut testbar, weil der Presenter ohne echte View geprüft werden kann.

Nachteil: Mehr Boilerplate: View-Interfaces, Presenter und Weiterleitungen müssen gepflegt werden.

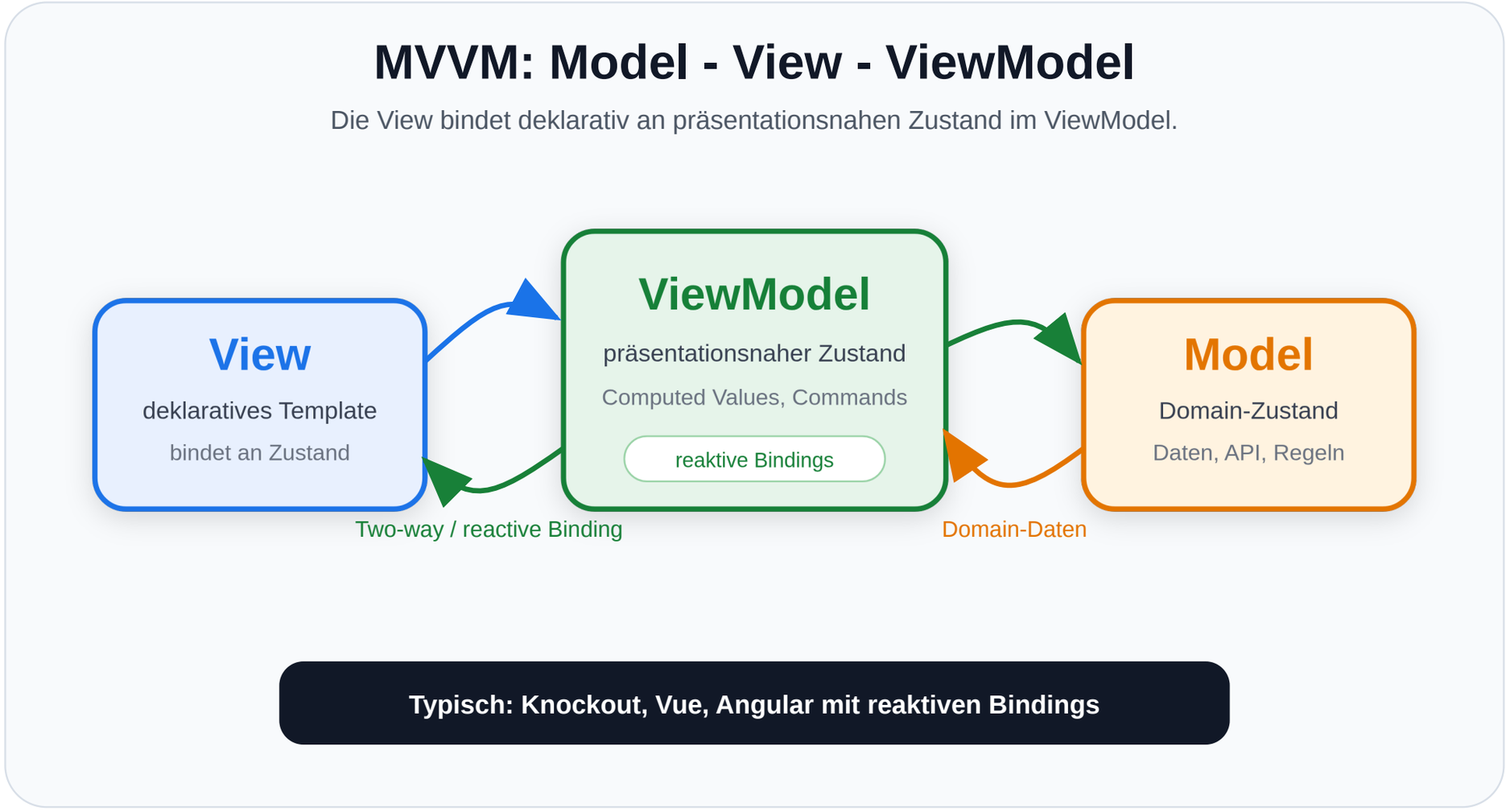
MVP: Die View bleibt passiv. Der Presenter liest aus der View, ändert das Model und sagt der View, was sie

anzeigen soll.

Speaker notes

Die View besteht hier aus dem sichtbaren HTML und einem dünnen Interface (`getName`, `showGreeting`). Dieses Interface enthält keine fachliche Entscheidung, sondern nur Zugriff auf die Oberfläche. Die Entscheidung, wann der Name gespeichert und welcher Text angezeigt wird, liegt im Presenter. Genau dadurch lässt sich der Presenter isoliert testen, indem `view.getName()` und `view.showGreeting()` durch Test-Doubles ersetzt werden.

MVVM: Model - View - ViewModel



MVVM verbindet die View deklarativ mit einem ViewModel. Änderungen am Zustand führen über Bindings automatisch zu UI-Aktualisierungen.

Speaker notes

MVVM ist eng mit deklarativen Bindings und reaktivem Zustand verbunden. Das ViewModel enthält Zustand, der direkt für die Darstellung vorbereitet ist, zum Beispiel formatierte Werte, abgeleitete Eigenschaften oder Commands. Frameworks wie Knockout haben MVVM sehr direkt umgesetzt; Vue und Angular enthalten viele MVVM-Ideen, auch wenn moderne Komponentenmodelle zusätzlich eigene Konzepte einführen.

MVVM am Mini-Beispiel

View

```
<input data-bind="name">
<button data-action="save">Speichern</button>
<p data-bind="greeting"></p>
```

Model

```
const model = { name: '' };
```

ViewModel

```
const viewModel = {
  name: '',
  get greeting() {
    return `Hallo ${this.name}`;
  },
  save() {
    model.name = this.name;
  }
};
```

`data-bind` steht hier für Framework-Bindings wie in Knockout, Vue oder Angular.

Vorteil: Deklarative Bindings reduzieren manuelle DOM-Aktualisierung.

Nachteil: Implizite Bindings können Debugging erschweren: Änderungen passieren automatisch und nicht immer sichtbar im Codepfad.

MVVM: Die View bindet deklarativ an das ViewModel. Der sichtbare Text ergibt sich aus Zustand und abgeleiteten Werten.

Speaker notes

MVVM verschiebt viel UI-Logik in präsentationsnahen Zustand. Die View muss nicht explizit wissen, wie `greeting` berechnet wird; sie bindet nur an einen Namen. Reaktive Frameworks aktualisieren die Anzeige automatisch, sobald sich abhängiger Zustand ändert. Das Model bleibt davon getrennt und enthält den fachlichen Zustand oder die serverseitig gespeicherten Daten.

Architekturfamilien moderner Frontends

Familie	Kernidee	Typische Verantwortung	Beispiele
MVC	Eingaben steuern einen Controller, der Model und View koppelt	View rendert, Controller verarbeitet Aktionen	Backbone, ältere AngularJS-Apps, serverseitige MVC-Frameworks
MVP	Presenter vermittelt explizit zwischen passiver View und Model	Presenter enthält fast die gesamte UI-Logik	GWT, viele testorientierte UI-Architekturen, Android historisch
MVVM	View bindet deklarativ an ein ViewModel	ViewModel hält präsentationsnahen Zustand	Vue, Knockout, Angular mit deutlichen MVVM-Elementen

MVC

- gut für klare Rollentrennung
- Controller wird in großen UIs schnell überladen

MVP

- stark testbar, weil die View passiv bleibt
- oft mehr Boilerplate durch Presenter-Schnittstellen

MVVM

- passt gut zu deklarativen Bindings
- heute das prägendste Denkmodell vieler Browser-Frameworks

Speaker notes

Diese drei Architekturfamilien sind keine starren Schubladen, sondern Denkmodelle für Zuständigkeiten. Moderne Browser-Frameworks sind oft Hybridformen. React ist nicht sauber MVC, MVP oder MVVM, sondern eher komponentenbasiert mit unidirektionalem Datenfluss. Vue liegt gedanklich oft nah an MVVM, weil Templates direkt an reaktiven Zustand gebunden werden. Alle drei Familien behandeln dasselbe Grundproblem: UI, Zustand und Benutzereingaben müssen so getrennt werden, dass große Frontends wartbar bleiben.

Moderne Frameworks: Familien als Bezugspunkt

Einordnung von gängigen Frameworks

- **Backbone**: nah an MVC
- **Knockout**: klassisches MVVM
- **Vue**: oft MVVM-artig mit Komponenten
- **Angular**: komponentenbasiert, mit MVVM-Elementen
- **React**: Komponentenmodell mit unidirektionalem Datenfluss

Was daraus folgt

- Frameworks sind selten „reines MVC“ oder „reines MVVM“
- sie übernehmen Ideen aus mehreren Familien
- entscheidend ist der Datenfluss: Wer hält Zustand, wer rendert, wer reagiert?
- Komponenten kapseln heute oft View + Verhalten + lokalen Zustand

Speaker notes

Ein Framework legt fest, wie Datenfluss, Komponentenstruktur, Rendering und Testbarkeit gedacht werden. React bevorzugt expliziten Datenfluss und lokale Komponentenlogik, Angular bringt stärkere Konventionen und Dependency Injection mit, Vue kombiniert reaktive Datenbindung mit Single-File Components. MVC, MVP und MVVM sind deshalb Bezugspunkte, keine exakten Etiketten für jedes moderne Framework.

Wrap-Up

- **HTML** definiert Struktur und Semantik — die Grundlage für Barrierefreiheit, SEO und maschinelle Verarbeitung.
- **JavaScript** macht den Browser programmierbar — über DOM, Events und asynchrone Kommunikation mit dem Server.
- **CSS** trennt Darstellung von Struktur — Flexbox und Grid lösen das Layout-Problem, Responsive Design das Geräteproblem.
- Die drei Technologien sind **eng verzahnt**: Der Browser verarbeitet sie in einer Pipeline (Parse → Render Tree → Layout → Paint), die JavaScript jederzeit beeinflussen kann.

Die nächste Vorlesung vertieft das **Protokoll** (HTTP) und das **API-Design** (REST), das die Brücke zwischen Frontend und Backend bildet.

Speaker notes

HTML, CSS und JavaScript sind jeweils eigene Spezifikationen mit separaten Gremien (WHATWG, CSS Working Group, TC39) und separaten Releases — aber im Browser laufen sie in einer gemeinsamen Pipeline. Das konzeptionelle Modell dieser Vorlesung — HTML als Strukturschicht, CSS als Constraint-Solver, JavaScript als Event-gesteuerte Laufzeit — ist unabhängig vom Framework-Ökosystem gültig und überdauert Framework-Wechsel. Vorlesung 04 wechselt auf die Protokollebene: HTTP definiert, wie Browser und Server kommunizieren; REST und andere API-Stile definieren, wie diese Kommunikation semantisch strukturiert wird. Die Fetch API, die in dieser Vorlesung auf der Client-Seite betrachtet wurde, ist die JavaScript-Schnittstelle zu diesem Protokoll.

Legacy-Quellen:

- `slides/04_02_WebEng_WebTechnologies-HTML-5.pptx`
- `slides/04_03_WebEng_WebTechnologies-CSS3.pptx`
- JavaScript-Inhalte sind neu erstellt (kein Legacy-Material vorhanden)